

HAWASA UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF COMPUTER SCIENCE

FINAL YEAR PROJECT I

PROJECT PROPOSAL

ADAPTIVE BUDGETING AND EXPENSE TRACKER

Advisor Name

Mr. Daniel S.

Submitted By:

Firew Kafalow	CoScW/0027/14
Abdurahman Mohammed	CoScW/0084/14
Mihret Hadish	CoScW/0041/14
Fentanesh Debesa	CoScE/0093/13
Mikiyas Teshale	CoScW/0043/14

May 2026

Hawassa, Ethiopia

DECLARATION

We, the undersigned, hereby declare that this project proposal titled **"Adaptive Budgeting and Expense Tracker"** submitted to the Department of Computer Science, Faculty of Informatics, Hawassa University, is our original work. We have duly acknowledged all sources of information used in this document.

We further declare that this work has not been submitted previously, in whole or in part, for any other degree or qualification at this or any other institution.

Submitted by:

Name	**ID Number**	**Signature**
Firew Kafalow	CoScW/0027/14	_____
Abdurahman Mohmmed	CoScW/0084/14	_____
Mihret Hadish	CoScW/0041/14	_____
Fentanesh Debesa	CoScW/0093/13	_____
Mikiyas Teshale	CoScW/0043/14	_____

Date: _____ (day/month/year)

ADVISOR APPROVAL

This project proposal titled **"Adaptive Budgeting and Expense Tracker"** has been prepared and submitted by the following students:

Name	**ID Number**
Firew Kafalow	CoScW/0027/14
Abdurahman Mohmmed	CoScW/0084/14
Mihret Hadish	CoScW/0041/14
Fentanesh Debesa	CoScW/0093/13
Mikiyas Teshale	CoScW/0043/14

under my guidance. I hereby confirm that I have reviewed this proposal and approve its submission to the Department of Computer Science, Faculty of Informatics, Hawassa University for evaluation.

Advisor Name: Mr. Daniel S.

Signature: _____

Date: _____(day/month/year)

ACKNOWLEDGEMENT

First and foremost, we would like to thank the Almighty God for giving us the strength, health, and wisdom to complete this project proposal successfully.

We extend our sincere gratitude to our advisor, **Mr. Daniel S.**, for his invaluable guidance, constructive feedback, and continuous support throughout the development of this proposal. His patience and expertise have been instrumental in shaping this work.

We are also grateful to the **Department of Computer Science, Faculty of Informatics, Hawassa University**, for providing the necessary resources and an enabling environment to carry out this project.

Our special thanks go to the students who participated in the interviews and requirement gathering sessions. Their honest responses and insights helped us understand the real-world challenges of financial management among university students.

Finally, we thank our families and friends for their unwavering encouragement, moral support, and understanding during the time we dedicated to this work.

Table of Contents

Chapter One Introduction.....1
1.1 Background of the Study.....1
1.2 Statement of the Problem.....2
1.2.1 Background of the Problem.....2
1.2.2 General Problem.....2
1.2.3 Specific Problem.....2
1.3 Objectives of the Project.....3
1.3.1 General Objective.....3
1.3.2 Specific Objectives.....3
1.4 Scope of the Study.....3
1.5 Limitation of the Study.....4
1.6 Methodology.....5
1.6.1 Data Collection Methodology.....5
1.6.2 System Analysis and Design Methodology.....6
1.6.3 System Implementation.....6
1.6.4 Testing and Deployment Methodology.....7
1.7 Development Environment.....8
1.8 System Requirements.....9
1.9 Feasibility Study.....9
1.9.1 Technical Feasibility.....9
1.9.2 Operational Feasibility.....9
1.9.3 Economic Feasibility.....10
Chapter Two Description of Existing and Proposed System.....11
2.1 Introduction of the Existing System.....11
2.2 Proposed System Description.....11
2.2.1 Core features of the proposed system include:.....11
2.3 Strengths of Existing System.....13

2.4 Weaknesses of Existing System.....13
Chapter Three System Features.....14
3.1 Introduction 14
3.2 Functional Requirements 14
3.3 Non-Functional Requirements 17
3.4 Analysis Models.....19
3.4.1 Use Case Descriptions.....19
3.4.2 Sequence Diagram.....33
3.4.3 Activity Diagram.....37
3.4.4 Class Diagram 42
Chapter Four System Design.....51
4.1 Introduction 51
4.2 System Architecture Design.....51
4.2.1 Architectural Layers.....52
4.2.2 Architectural Workflow 52
4.2.3 Advantages of the Proposed Architecture 53
4.3 System Architecture Diagram.....53
4.4 System Component Design.....54
4.4.1 Authentication Component 55
4.4.2 Expense Management Component 55
4.4.3 Debt Tracking Component.....55
4.4.4 Dashboard Component.....56
4.4.5 Adaptive Features Component.....56
4.4.6 Synchronization Component.....56
4.5 Database Design Overview.....57
4.5.1 Entity Relationship Design 57
4.5.2 Relationships and Integrity 60
4.6 Offline Synchronisation 60

4.7 User Interface Design.....60
4.7.1 Main Screen Specifications.....61
4.8 API Design 63
4.9 Security Design 64

List of Figures

Figure 3-1: Use Case Diagram of the Adaptive Budgeting and Expense Tracker 20
Figure 3-2: Add Expense Sequence Diagram 33
Figure 3-3: Debt Recording Sequence Diagram 34
Figure 3-4: Background Synchronization Sequence Diagram 34
Figure 3-5: Dashboard Analytics and Prediction Sequence Diagram 35
Figure 3-6: Reminder Detection Sequence Diagram 36
Figure 3-7: Password Reset Sequence Diagram 36
Figure 3-8: Add Expense Workflow (Activity Diagram) 37
Figure 3-9: Debt Recording Workflow (Activity Diagram) 38
Figure 3-10: Background Synchronization Workflow (Activity Diagram) 39
Figure 3-11: Dashboard Analytics and Prediction Workflow (Activity Diagram) 40
Figure 3-12: Password Reset Workflow (Activity Diagram) 41
Figure 3-13: Reminder Detection Workflow (Activity Diagram) 42
Figure 3-14: Class Diagram of the Adaptive Budgeting and Expense Tracker 50
Figure 4-1: System Architecture Diagram 53
Figure 4-2: System Component Diagram 54
Figure 4-3: Entity-Relationship Diagram (ERD) 59
Figure 4-4: User Interface Screens 62

List of Tables

Table 1-1: Data Collection Methods and Their Purpose 5
Table 1-2: System Implementation Technologies and Their Roles.....6
Table 1-3: Testing Levels and Descriptions 7
Table 1-4: Development Environment Components and Technologies 8
Table 1-5: Mobile Application System Requirements 9
Table 3-1: Functional Requirements – User Management 14
Table 3-2: Functional Requirements – Expense Logging 14
Table 3-3: Functional Requirements – Informal Debt Tracking 15
Table 3-4: Functional Requirements – Spending Insights and Visualization.....15
Table 3-5: Functional Requirements – Adaptive Features 15
Table 3-6: Functional Requirements – Notifications and Reminders 16
Table 3-7: Functional Requirements – Offline Support and Synchronization.....16
Table 3-8: Functional Requirements – Password Management 16
Table 3-9: Functional Requirements – Data Backup and Recovery...16
Table 3-10: Functional Requirements – Category Customization 16
Table 3-11: Functional Requirement – Integrated Dashboard 17
Table 3-12: Non-Functional Requirements – Performance.....17
Table 3-13: Non-Functional Requirements – Security 17
Table 3-14: Non-Functional Requirements – Usability.....18
Table 3-15: Non-Functional Requirements – Reliability and Availability 18
Table 3-16: Non-Functional Requirements – Maintainability.....18
Table 3-17: Non-Functional Requirements – Platform Compatibility 19
Table 3-18: Use Case UC01 – Register 21
Table 3-19: Use Case UC02 – Login 21
Table 3-20: Use Case UC03 – Reset Password 21
Table 3-21: Use Case UC04 – Logout 22
Table 3-22: Use Case UC05 – Update Profile 22
Table 3-23: Use Case UC06 – Set Monthly Budget.....22

Table 3-24: Use Case UC07 – Add Expense 23
Table 3-25: Use Case UC08 – Edit Expense 23
Table 3-26: Use Case UC09 – Delete Expense.....23
Table 3-27: Use Case UC10 – View Expenses.....24
Table 3-28: Use Case UC11 – Filter Expenses.....24
Table 3-29: Use Case UC12 – Search Expenses by Keyword 24
Table 3-30: Use Case UC13 – Record Borrowed Money 25
Table 3-31: Use Case UC14 – Record Lent Money 25
Table 3-32: Use Case UC15 – Settle Debt.....25
Table 3-33: Use Case UC16 – Partial Repayment.....26
Table 3-34: Use Case UC17 – View Debts.....26
Table 3-35: Use Case UC18 – Search Debts by Name.....26
Table 3-36: Use Case UC19 – Edit Debt 27
Table 3-37: Use Case UC20 – Delete Debt 27
Table 3-38: Use Case UC21 – View Pie Charts by Category 27
Table 3-39: Use Case UC22 – View Spending Trends 28
Table 3-40: Use Case UC23 – Compare Spending vs Budget 28
Table 3-41: Use Case UC24 – View Highest Spending Category 28
Table 3-42: UC41 – View Dashboard.....29
Table 3-43: UC30 – Set Manual Reminder 29
Table 3-44: UC31 – Send Online Notification 29
Table 3-45: UC32 – Queue Offline Notification 30
Table 3-46: UC33 – Store Data Locally (SQLite) 30
Table 3-47: UC34 – Operate Fully Offline.....30
Table 3-48: UC35 – Sync Data when Online 31
Table 3-49: UC36 – Manual Backup 31
Table 3-50: UC37 – Auto Backup (daily).....31
Table 3-51: UC38 – Restore Data.....32
Table 3-52: UC39 – Create Personal Category.....32
Table 3-53: UC40 – Edit/Delete Personal Category.....32
Table 3-54: Overview of Main Classes.....43
Table 3-55: Class User – Attributes 43
Table 3-56: Class User – Methods 44
Table 3-57: Class Category – Attributes.....44
Table 3-58: Class Category – Methods.....45

Table 3-59: Class Expense – Attributes 45
Table 3-60: Class Expense – Methods 46
Table 3-61: Class Debt – Attributes 46
Table 3-62: Class Debt – Methods 47
Table 3-63: Class Reminder – Attributes 47
Table 3-64: Class Reminder – Methods 48
Table 3-65: Class SyncLog – Attributes 48
Table 3-66: Class SyncLog – Methods 48
Table 3-67: Relationships Summary (Cardinalities) 49
Table 3-68: Mapping of Classes to Functional Requirements 49
Table 4-1: Advantages of the Proposed Architecture 53
Table 4-2: User Entity Attributes 57
Table 4-3: Category Entity Attributes 57
Table 4-4: Expense Entity Attributes 58
Table 4-5: Debt Entity Attributes 58
Table 4-6: Reminder Entity Attributes 58
Table 4-7: SyncLog Entity Attributes 59
Table 4-8: Authentication API Endpoints 63
Table 4-9: Expense Management API Endpoints 63
Table 4-10: Debt Management API Endpoints 63
Table 4-11: Synchronization API Endpoints 64

LIST OF ABBREVIATIONS

Abbreviation	Meaning
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
APK	Android Package Kit
CRUD	Create, Read, Update, Delete
ERD	Entity-Relationship Diagram
FR	Functional Requirement
GB	Gigabyte
Git	Version control system
GitHub	Git hosting platform
GORM	Go Object-Relational Mapping library
GSMA	GSM Association (mobile industry body)
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
iOS	iPhone Operating System (Apple)
JSON	JavaScript Object Notation
MB	Megabyte
ML	Machine Learning
NFR	Non-Functional Requirement
OOP	Object-Oriented Programming
ORM	Object-Relational Mapping
OS	Operating System
RAM	Random Access Memory
REST	Representational State Transfer
SDK	Software Development Kit
SQL	Structured Query Language
SQLite	Embedded relational database engine
TLS	Transport Layer Security
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modeling Language
VS Code	Visual Studio Code
XSS	Cross-Site Scripting

Chapter One
Introduction

In an era of increasing financial complexity, managing personal finances effectively is a critical life skill. However, students and young professionals in Ethiopia face unique challenges due to cash-based transactions and informal lending networks. Existing financial applications are designed for users with formal banking relationships and do not address these needs. This project proposes the development of an Adaptive Budgeting and Expense Tracker, a cross-platform, offline capable application that adapts to user behaviour to provide intelligent spending analysis, alerts, recommendations, predictions, and personalized experiences—without requiring internet connectivity or complex external integrations.

1.1 Background of the Study

Financial management is a critical life skill that enables individuals to track income, control spending, and plan for future needs. However, for students and young professionals, managing personal finances presents unique challenges. These individuals often operate primarily with cash transactions and informal peer-to-peer lending, where money is borrowed from or lent to friends, family, and classmates. Unlike formal banking systems, these transactions are rarely documented, leading to forgotten debts, unrecorded expenses, and financial stress.

According to Lusardi and Mitchell (2014), young adults consistently report higher levels of financial anxiety compared to older generations, largely due to a lack of accessible tools tailored to their specific financial behaviours. While numerous financial applications exist in the market—such as Mint, YNAB (You Need a Budget), and Expensify—these platforms are primarily designed for individuals with established bank accounts, credit cards, and investment portfolios (Malaquias & Hwang, 2016). They often assume a level of financial complexity that does not align with the daily realities of student life.

In Ethiopia and similar contexts, the situation is further compounded by a culture of informal lending within social networks (Worku, 2022). Students frequently lend money to peers for meals, transportation, or academic materials and these debts are often forgotten or lead to misunderstandings. Existing applications do not address this specific need, leaving young

people without a simple, intuitive tool to manage both their personal expenses and interpersonal debts.

The advent of mobile technology and the increasing penetration of smartphones in Ethiopia present an opportunity to bridge this gap (GSMA, 2023). A lightweight, cross-platform application that focuses on practical daily financial behaviours—expense logging, debt tracking, and spending visualization—can empower students and young professionals to take control of their finances. Such a tool can reduce financial stress, minimize conflicts over unpaid debts, and help users build healthy financial habits from an early age.

1.2 Statement of the Problem

1.2.1 Background of the Problem

Students and young professionals in Ethiopia operate in a cash-based economy where informal lending among peers is a daily reality (Collins et al., 2009). Without formal documentation, expenses often go unrecorded, and money lent to friends or family is frequently forgotten. Previous research shows that low financial literacy and the absence of accessible, context appropriate tools exacerbate these issues (Lusardi & Mitchell, 2014).

Anchor

A 2022 study of Ethiopian university students found that over 68% of respondents had experienced a forgotten debt or a misunderstanding over borrowed money within the past six months (Worku, 2022). This high prevalence demonstrates the need for a practical solution.

1.2.2 General Problem

The general problem is that unrecorded cash transactions and informal debts lead to financial mismanagement, interpersonal conflicts, financial anxiety, and missed opportunities for saving among young adults in Ethiopia.

1.2.3 Specific Problem

No existing financial application provides an integrated, offline-capable solution for manually logging daily expenses and tracking peer-to-peer debts tailored to Ethiopian students and young professionals. Consequently, the target population lacks a simple tool to:

1. Record cash expenses by category.
2. Track money lent to or borrowed from friends.
3. Receive adaptive alerts for budget overruns, overdue debts, or unusual spending.

4. Obtain personalized recommendations and spending predictions.
5. Visualize spending patterns without internet connectivity.

This gap directly motivates the development of the proposed Adaptive Budgeting and Expense Tracker.

1.3 Objectives of the Project

1.3.1 General Objective

The general objective of this project is to design and develop a cross platform adaptive budgeting and expense tracker that enables students and young professionals to effectively manage daily expenses, track informal debts, and build long term financial awareness.

1.3.2 Specific Objectives

To achieve the general objective, the following specific objectives are set:

- To design a user profile system that enables registration, login, and customization of spending preferences.
- To develop a module for recording, editing, and categorizing daily expenses into predefined categories.
- To implement a peer-to-peer debt tracking feature for recording, updating, and settling borrowed and lent amounts.
- To provide visual summaries of expenses by category and time using charts and graphs.
- To integrate notification and reminder functionality for pending debt settlements and recurring expenses.
- To enable offline data storage and automatic synchronization when internet connectivity is available.
- To implement adaptive features including spending analysis, alerts, recommendations, budget prediction, personalized category suggestions, search, and adaptive reminders using rule-based logic.

1.4 Scope of the Study

This project focuses on developing a practical financial tracking tool tailored for students and young professionals. The scope is defined as follows to ensure the project remains manageable and achievable within the timeline.

- User registration, login, and profile management

- Expense logging with categorization (food, transport, entertainment, education, etc.)
- Informal debt and loan tracking (money owed to and borrowed from others)
- Spending summaries and trend visualization using charts and graphs
- Push notifications and reminders for debts and recurring expenses
- Offline-first mobile logging with data synchronization
- Cross-platform mobile support using Flutter for Android and iOS
- Secure data storage using a PostgreSQL database
- Automated rule-based features

The following features are excluded from this project to maintain focus on the core financial tracking needs of the target users and due to scope, time, and resource constraints:

- Web-based dashboard or desktop browser access
- Integration with formal banking systems or financial APIs
- Investment advice, stock tracking, or financial planning tools
- Credit scoring, lending marketplace, or loan origination
- Tax calculation, invoice generation, or business accounting features
- Social media integration or community feeds
- Multi-currency support (initially limited to Ethiopian Birr)
- Machine learning or artificial intelligence models

1.5 Limitation of the Study

The following limitations are acknowledged for this project:

- Limited User Testing – The system will be tested with a limited sample of students at Hawassa University. This sample may not represent all demographics across different universities or age groups. Consequently, the findings may not be fully generalizable.
- No Bank Integration – The application will not connect to bank accounts or mobile money services, relying entirely on manual user input. This limits automation but keeps the system simple and accessible for cash-based users.
- Time Constraints – The project must be completed within a single academic semester. This may limit the depth of features implemented and prevent inclusion of advanced analytics or group debt management.
- Rule-based Intelligence Only - All adaptive features will use deterministic rules and statistics, not machine learning, which may limit predictive accuracy.

1.6 Methodology

This section describes the systematic approach that will be followed to gather requirements, design, implement, test, and deploy the Adaptive Budgeting and Expense Tracker. The project will adopt an Agile development approach to support iterative development, continuous refinement, and user-centred system improvement throughout the project lifecycle.

1.6.1 Data Collection Methodology

To identify the functional and non-functional requirements of the proposed system, multiple data collection techniques were used. These techniques helped provide insight into users' financial management practices, challenges, and expectations from a budgeting application.

The following methods were employed during requirement gathering.

Table 1-1: Data Collection Methods and Their Purpose

Method	**Purpose**	**Target**
Interview	To understand current financial management practices and pain points	Students at Hawassa University
Observation	To observe how users currently track expenses and manage debts	Selected participants
Document Analysis	To review existing financial apps and identify gaps	Mint, YNAB, Expensify, and local alternatives

Informal interviews were conducted with students from different academic years and financial backgrounds to obtain diverse perspectives regarding budgeting behaviour, expense tracking habits, debt management practices, and challenges associated with existing applications.

Observation techniques were also used to examine how users currently manage their finances. Observations helped identify usability limitations and the need for simplified financial tracking features.

In addition, document analysis was conducted on existing budgeting and expense tracking applications such as Mint, YNAB, Expensify, and locally available alternatives. The analysis focused on identifying feature gaps relevant to the target users, particularly limitations related

to offline functionality, informal debt tracking, localized budgeting practices, and adaptive financial recommendations.

The information collected through these methods was analyzed and used to define the project scope, system requirements, and major functional components of the proposed system.

1.6.2 System Analysis and Design Methodology

The system development will follow the Agile Development Methodology, which emphasises iterative development, continuous feedback, and incremental delivery. This approach is chosen because it allows the team to respond to changing requirements and incorporate user feedback throughout the development lifecycle. The methodology will be implemented through the following phases

- Requirements Analysis: Gathering and documenting functional and non-functional requirements.
- Design: Creating system architecture, database schema, and user interface prototypes.
- Development: Implementing features in iterative sprints.
- Testing: Conducting unit, integration, and user acceptance testing.
- Deployment: Releasing the application for Android and iOS mobile platforms.

1.6.3 System Implementation

The system will be implemented using cross-platform architecture with separate frontend, backend, and database components. The implementation will follow an object-oriented programming (OOP) approach to ensure modularity, reusability, and maintainability.

The table below summarises the key implementation technologies and their roles.

Table 1-2: System Implementation Technologies and Their Roles

Component	**Technology**	**Purpose**
Mobile Frontend	Flutter (Dart)	Build iOS and Android apps from a single codebase; provide offline-first capability with local SQLite storage.
Backend API	Golang (REST API)	Handle business logic, authentication, data validation, and synchronisation.
Database	PostgreSQL	Store user accounts, expenses, debts, and reminders persistently on the server.

The development order will prioritise core functionalities first, as follows:

- Sprint 1: User registration, login, and profile management.

- Sprint 2: Expense logging module (adds, edit, delete, view expenses with categories).
- Sprint 3: Informal debt tracking module (record lent/borrowed amounts, settle debts, view pending debts).
- Sprint 4: Spending insights (charts and graphs) and reminder/notification system.
- Sprint 5: Offline functionality for mobile (local SQLite storage) and synchronisation with the server.
- Sprint 6: Adaptive features (spending analysis, alerts, recommendations, prediction, personalization, smart search, smart reminders)
- Sprint 7: Final integration, testing, and bug fixing.

All code will be managed using Git (GitHub) for version control. The team will follow consistent coding conventions and document the code inline to facilitate future maintenance.

1.6.4 Testing and Deployment Methodology

Testing Methodology

To ensure the quality, reliability, and usability of the Adaptive Budgeting and Expense Tracker, a systematic testing strategy will be applied at multiple levels.

The following testing levels will be conducted:

Table 1-3: Testing Levels and Descriptions

Test Level	**Description**
Unit Testing	Testing individual components (Flutter widgets components, Go functions)
Integration Testing	Testing API endpoints and database interaction
System Testing	Testing the complete application across mobile platforms
User Acceptance Testing	Testing with target users to validate requirements

All defects will be logged and prioritised (critical, major, minor).

Deployment Methodology

The deployment will follow a phased rollout approach:

- Local Development Environment – The team will develop and test the system on local machines using emulators (Android Studio).
- Staging Environment – The backend API will be deployed to a cloud server for integration testing. The mobile app will be distributed via TestFlight (iOS) and internal APK sharing (Android).

- Production Deployment – After successful UAT, the mobile app will be packaged for manual installation.
- Offline Synchronisation – The mobile app will be designed to work offline; data will be stored locally in SQLite and automatically synchronised with the server when an internet connection is restored.

A rollback plan will be in place: if critical issues are discovered post-deployment, the system will revert to the previous stable version while fixes are applied.

1.7 Development Environment

The development environment consists of the hardware and software tools that will be used to build, test, and manage the Adaptive Budgeting and Expense Tracker. The table below lists the key components.

Table 1-4: Development Environment Components and Technologies

Component	**Technology**	**Purpose**
IDE	Visual Studio Code/Android Studio	Writing and debug code
Database Management	pgAdmin/DBeaver	Manage PostgreSQL databases (tables, queries, backups).
Version Control	Git (GitHub)	Track code changes, collaborate, and manage releases
API Testing	Postman	Test REST API endpoints manually and automate API tests
UI Prototyping	Figma	Design wireframes and mockups before implementation
Local Database	SQLite	Store data offline on mobile devices during development
Local Development OS	Windows 10/11	Workstations for the development team

All development will follow a consistent workflow: code → commit to GitHub → run unit tests → integrate → deploy to staging. The team will use feature branches and pull requests to review code changes before merging.

1.8 System Requirements

The following system requirements specify the minimum and recommended configurations for end-users to run the Adaptive Budgeting and Expense Tracker effectively. These requirements apply to mobile application (Android/iOS).

Table 1-5: Mobile Application System Requirements

Platform	**Minimum Requirements**	**Recommended Requirements**
Android	Android 8.0 (Oreo) or later, 2 GB RAM, 50 MB free storage	Android 11 or later, 4 GB RAM, 100 MB free storage
iOS	iOS 13.0 or later, 2 GB RAM, 50 MB free storage	iOS 15 or later, 4 GB RAM, 100 MB free storage
Internet	Intermittent connection (offline mode supported); required only for synchronisation and initial login	Stable Wi-Fi or mobile data for synchronisation

These requirements will be validated during the testing phase to ensure smooth operation across target devices and platforms.

1.9 Feasibility Study

1.9.1 Technical Feasibility

The project is technically feasible because:

- The team has foundational knowledge in Flutter, Golang, and PostgreSQL.
- All required development tools are open source and freely available.
- Extensive documentation and community support exist for the chosen technologies.
- Potential technical challenges include offline synchronization and real time notifications, but these can be addressed through research and iterative development.

1.9.2 Operational Feasibility

The proposed system will be simple and intuitive, requiring no prior financial knowledge. It will work offline, making it suitable for students with limited internet access. The system will not disrupt existing manual practices but will complement them. Therefore, the system is operationally feasible.

1.9.3 Economic Feasibility

The project will use only open-source software (Flutter, Golang, PostgreSQL) and free development tools (VS Code, Android Studio, Git). No hardware purchases are required as development will be done on existing team laptops. Hence, the project is economically feasible within a zero-budget constraint.

Chapter Two

Description of Existing and Proposed System

2.1 Introduction of the Existing System

Currently, students and young professionals at Hawassa University and its surrounding community manage their daily finances using entirely manual and informal methods. No digital tool is consistently used to record expenses or track debts.

At present, most students rely on a combination of mental tracking, handwritten notebooks, and spreadsheets to record expenses and debts. Common practices include:

- Mental tracking – Many students attempt to remember what they have spent and whom they owe. This approach becomes unreliable when multiple transactions occur.
- Notebooks or diaries – Some students keep a small notebook where they write down expenses and loans. Entries are often sporadic and lack categorisation.
- Mobile notes – A few uses the notes app on their phones to jot down debts, but there is no structured way to view summaries or receive reminders.
- Word of mouth – Debts between friends are often settled based on memory alone, leading to occasional disputes or forgotten amounts.

No single tool is used consistently, and none provides automated insights or reminders. The existing system is entirely manual and informal, relying on the user's memory or ad-hoc written records.

2.2 Proposed System Description

The proposed Adaptive Budgeting and Expense Tracker is a mobile-first application designed to address the weaknesses of the existing manual approach while preserving the simplicity and social context of informal finance. It will be available as a mobile app (Flutter), backed by a Golang REST API and a PostgreSQL database.

2.2.1 Core features of the proposed system include:

- User-friendly registration and profile management – Users will create an account with basic information and can optionally set spending preferences.
- Expense logging – A simple interface will allow users to record an expense with a description, amount, category, and date. Edits and deletions will be supported.

- Informal debt tracking – Users will record money lent to a contact or money owed to someone else. Each debt record will include the person's name (or phone number), amount, date, and a status (pending or settled). Notifications will be sent for pending debts.
- Spending insights – Charts and graphs (by category, by time) will help users visualise their spending habits.
- Reminders – Users will set optional reminders for recurring expenses or for debt settlement deadlines.
- Offline support – The mobile app will store data locally using SQLite and synchronise with the server when a connection is available.
- Security - The system will be secure, with user passwords hashed and data transmitted over HTTPS. It will not integrate with banks or mobile money services, maintaining a clear boundary for manual entry.

Adaptive Features

The system will include the following adaptive features to provide an intelligent experience without machine learning. All features are implemented using deterministic rules and statistics, ensuring they work offline and require no external APIs.

- Smart Spending Analysis – Automatically compares each category's total to overall spending.
- Smart Alerts & Notifications – Uses IF-THEN rules to notify users when spending exceeds a budget, a debt is overdue, or an unusual amount is detected.
- Smart Recommendations – Identifies the user's highest spending category and suggests a reduction with a template message.
- Budget Prediction – Moving average: (average of last 7 days) × (days in month) to predict monthly spending.
- Personalized Experience – Stores the user's frequently used categories and preferred patterns in the database. Auto-suggests categories when adding expenses.
- Smart Search / Filtering – Enables quick finding of expenses by name or debts by person using SQL LIKE queries with partial matching.
- Smart Reminders – Detects recurring transactions (same amount, same day of month/week).

All adaptive features are implemented using rule-based logic and statistics.

2.3 Strengths of Existing System

Despite its informal nature, the current manual approach has certain strengths:

- Simplicity – No learning curve is required; users already know how to write a note or remember a debt.
- No technology dependency – It works regardless of internet connectivity or device availability.
- Social accountability – Debts are often remembered because they involve personal relationships, which creates a social incentive to repay.

These strengths indicate that any digital replacement must be equally simple and should complement, not complicate the user's social habits.

2.4 Weaknesses of Existing System

The weaknesses of the current manual system are significant and directly motivate the need for a digital solution:

- Unreliable record-keeping – Mental tracking fails when the number of transactions grows, and written notes are easily lost.
- No spending visibility – Users cannot see aggregated spending patterns or identify where their money goes.
- Forgotten debts – Without reminders, both borrowers and lenders often forget obligations, leading to financial loss or awkward conversations.
- No categorisation – Expenses are not grouped, making it impossible to analyse trends (e.g., how much is spent on food vs. transport).
- Time-consuming – Manually summing up expenses or calculating balance is tedious and error-prone.
- Lack of backup – If a notebook is misplaced or a phone is lost, all records disappear.
- No adaptive intelligence – Users receive no alerts, recommendations, or predictions to improve their financial decisions.

These weaknesses highlight the need for a tool that provides reliable, categorised, and synchronised expense and debt tracking while retaining the simplicity and social awareness of informal methods.

Chapter Three

System Features

3.1 Introduction

This chapter defines the complete feature set of the Adaptive Budgeting and Expense Tracker by specifying both functional and non-functional requirements. It translates the user needs identified in earlier chapters into concrete system behaviours, covering user management, expense and debt tracking, adaptive intelligence, notifications, offline support, password recovery, data backup, category customization, and an integrated dashboard.

In addition, the chapter presents a set of analysis models—use case diagrams, sequence diagrams, activity diagrams, and class diagrams—that describe the system's structure, dynamic interactions, and workflows. These models serve as a blueprint for the design and implementation phases, ensuring that all stakeholders have a shared understanding of how the system will operate.

3.2 Functional Requirements

The functional requirements define the specific behaviours the system must support. They are grouped by feature area.

Table 3-1: Functional Requirements – User Management

ID	Requirement
FR-01	The system shall allow a new user to register using an email address and password.
FR-02	The system shall require password confirmation during registration.
FR-03	The system shall hash user passwords before storing them.
FR-04	The system shall allow registered users to log in.
FR-05	The system shall provide a password reset option via email.
FR-06	The system shall allow users to update profile information.
FR-07	The system shall allow users to set a monthly budget limit.

Table 3-2: Functional Requirements – Expense Logging

ID	Requirement
FR-08	The system shall allow users to add expenses (amount, category, date, description).
FR-09	The system shall provide predefined categories.

FR-10	The system shall allow editing expenses.
FR-11	The system shall allow deletion of expenses with confirmation.
FR-12	The system shall display a list of all expenses sorted by date (most recent first).
FR-13	The system shall allow users to filter expenses by date range and category.

Table 3-3: Functional Requirements – Informal Debt Tracking

ID	**Requirement**
FR-14	The system shall allow users to record money borrowed from another person.
FR-15	The system shall allow users to record money lent to another person.
FR-16	The system shall display “I owe” and “Others owe me”.
FR-17	The system shall allow marking debts as settled.
FR-18	The system shall support partial repayment.
FR-19	The system shall allow editing and deleting debts.
FR-20	The system shall support searching debts by name.

Table 3-4: Functional Requirements – Spending Insights and Visualization

ID	**Requirement**
FR-21	The system shall generate pie charts by category.
FR-22	The system shall generate spending trend graphs.
FR-23	The system shall compare spending with budget.
FR-24	The system shall display highest spending category.

Table 3-5: Functional Requirements – Adaptive Features

ID	**Requirement**
FR-25	Smart Spending Analysis: The system shall analyse spending by category and highlight any category that exceeds a predefined threshold.
FR-26	Smart Alerts: The system shall generate alerts when predefined financial conditions or unusual spending patterns are detected.
FR-27	Smart recommendations: The system shall identify the highest spending category and display a contextual recommendation message.
FR-28	Budget Prediction: The system shall estimate spending using a moving average based on the previous expenses. The predicted value shall be displayed on the dashboard.
FR-29	Personalized Experience: The system shall track frequently used categories and prioritize the top three categories when users add new expenses.

FR-30	Smart Search: The system shall allow users to search expenses and debts using keyword-based partial matching.
FR-31	Smart Reminders: The system shall detect recurring transactions based on repeated spending patterns and prompt users to create reminders.

Table 3-6: Functional Requirements – Notifications and Reminders

ID	Requirement
FR-32	The system shall allow manual reminders.
FR-33	The system shall send notifications when online.
FR-34	The system shall queue notifications when offline.

Table 3-7: Functional Requirements – Offline Support and Synchronization

ID	Requirement
FR-35	The system shall store data locally using SQLite.
FR-36	The system shall operate offline fully.
FR-37	The system shall synchronize data when internet is available.

Table 3-8: Functional Requirements – Password Management

ID	Requirement
FR-38	The system shall allow a user to request a password reset.
FR-39	The system shall allow the user to set a new password after verification.

Table 3-9: Functional Requirements – Data Backup and Recovery

ID	Requirement
FR-40	Manual Backup: The system shall allow users to manually create local backups of expenses, debts, and reminders.
FR-41	Automatic Backup: The system shall automatically create local backups when the application is closed or the device is idle.
FR-42	Data Restoration: The system shall allow users to restore data from the latest local backup or synchronized server backup after reinstallation or data loss.

Table 3-10: Functional Requirements – Category Customization

ID	Requirement
FR-43	The system shall allow users to create new personal expense categories.
FR-44	The system shall allow users to edit or delete their personal categories.

Table 3-11: Functional Requirement – Integrated Dashboard

ID	Requirement
FR-45	The system shall provide a dashboard screen summarizing: current month's spending vs. budget (if set), the most recent expenses, any active alerts, and the current budget prediction.

3.3 Non-Functional Requirements

The non-functional requirements define the quality attributes the system must satisfy, including performance, security, usability, reliability, maintainability, and platform compatibility.

Table 3-12: Non-Functional Requirements – Performance

ID	Requirement
NFR-01	The mobile application shall launch within 3 seconds on a device meeting the minimum requirements (Android 8.0 / iOS 13.0).
NFR-02	All local operations (adding, editing, deleting expenses/debts) shall complete within 500 milliseconds.
NFR-03	Synchronization with the server shall not block the user interface.
NFR-04	The application shall consume no more than 50 MB of additional storage per 1,000 expense records.
NFR-05	Charts and graphs (e.g., pie chart, bar chart) shall render within 1 second for up to 500 expense entries.

Table 3-13: Non-Functional Requirements – Security

ID	Requirement
NFR-06	All user passwords shall be hashed using bcrypt (or a comparable algorithm) before storage.
NFR-07	All communication between the mobile app and the backend API shall be encrypted using HTTPS (TLS 1.2 or higher).
NFR-08	The system shall lock the user session after 15 minutes of inactivity and require re-authentication.
NFR-09	The backend API shall validate all inputs to prevent SQL injection and cross-site scripting (XSS) attacks.
NFR-10	Local database encryption is deferred to future versions.

Table 3-14: Non-Functional Requirements – Usability

ID	Requirement
NFR-11	The application shall require no more than three taps/clicks to perform any primary action (add expense, add debt, view insights).
NFR-12	All buttons and interactive elements shall have clear labels and consistent placement across screens.
NFR-13	The application shall provide visual feedback (e.g., loading indicator, success message) for all user actions lasting more than 1 second.
NFR-14	The default font size shall be at least 14sp for body text and 18sp for headings, supporting readability on small screens.
NFR-15	Error messages shall be written in plain language (no technical jargon) and suggest corrective actions.

Table 3-15: Non-Functional Requirements – Reliability and Availability

ID	Requirement
NFR-16	The mobile application shall not crash or lose data when switching between online and offline modes.
NFR-17	In the event of an unexpected shutdown (e.g., low battery, app crash), all already saved data shall persist and remain uncorrupted.
NFR-18	The backend API shall have an uptime target of 99% during the testing and demonstration period.
NFR-19	The system shall handle at least 100 concurrent users during testing without performance degradation beyond NFR-01 to NFR-05.

Table 3-16: Non-Functional Requirements – Maintainability

ID	Requirement
NFR-20	The source code shall follow consistent naming conventions and include inline comments for all non-trivial functions.
NFR-21	The backend API shall be documented using OpenAPI (Swagger) or a similar standard.
NFR-22	The database schema shall be normalized to at least third normal form (3NF) to avoid redundancy and simplify future changes.
NFR-23	The system shall use environment variables for all configuration settings.

Table 3-17: Non-Functional Requirements – Platform Compatibility

ID	Requirement
NFR-24	The Android version of the application shall run on Android 8.0 (Oreo) and all later versions.
NFR-25	The iOS version of the application shall run on iOS 13.0 and all later versions.
NFR-26	The application shall support screen sizes from 4.0 inches to 6.9 inches (phones and small tablets) without broken layouts.
NFR-27	The application shall support both portrait and landscape orientations.

3.4 Analysis Models

3.4.1 Use Case Descriptions

This section presents the detailed use case descriptions of the Adaptive Budgeting and Expense Tracker. The use cases are grouped into functional modules for clarity and alignment with system features. Each use case includes the use case ID, actor, precondition, postcondition, and basic flow.

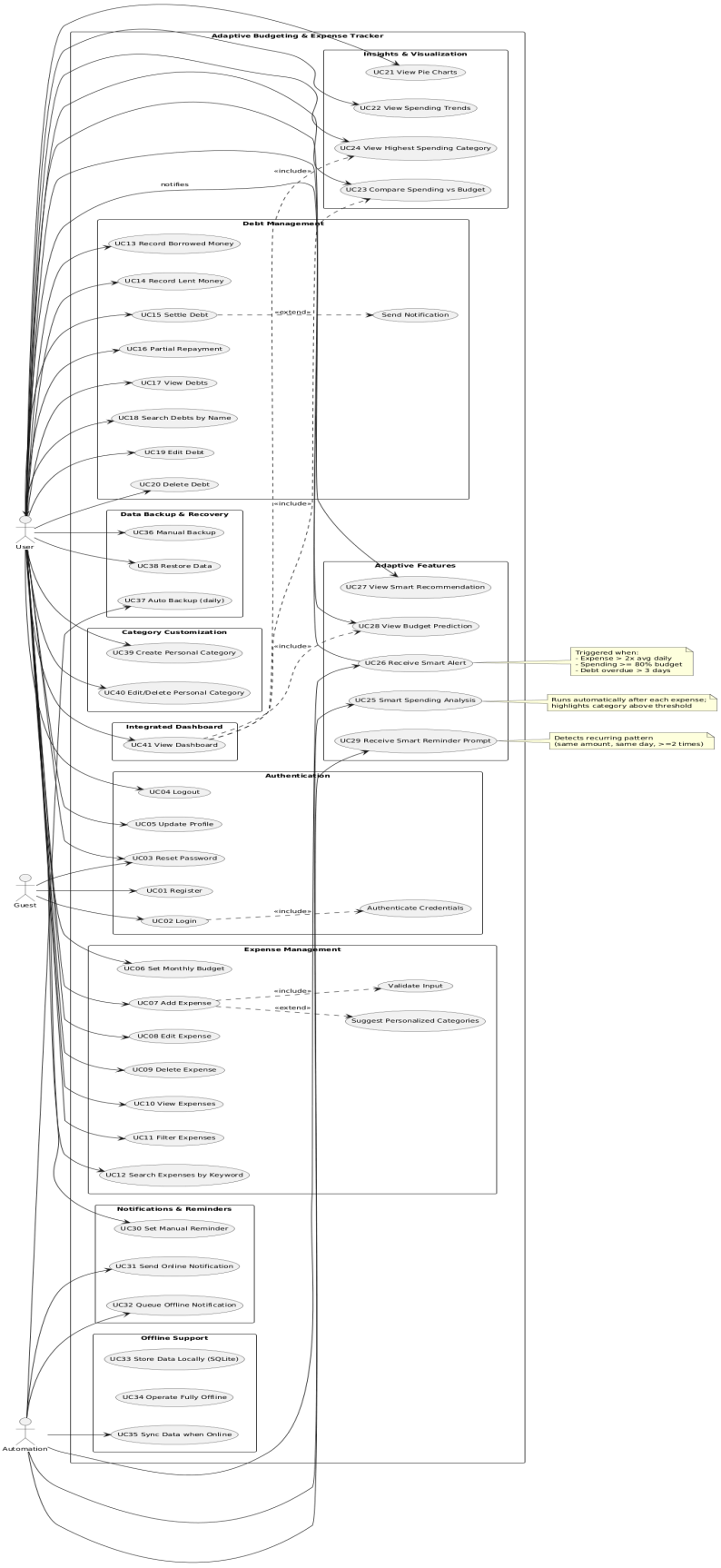


Figure 3-1: Use Case Diagram of the Adaptive Budgeting and Expense Tracker

Authentication

Table 3-18: Use Case UC01 – Register

Field	Description
Use Case ID	UC01
Use Case Name	Register
Actor	Guest
Precondition	No existing account with the provided email.
Postcondition	A new user account is created.
Basic Flow	1. User enters email and password. 2. System validates input (email format, password strength). 3. System creates account and stores hashed password. 4. Confirmation message is shown.

Table 3-19: Use Case UC02 – Login

Field	Description
Use Case ID	UC02
Use Case Name	Login
Actor	Guest
Precondition	User account exists.
Postcondition	User is authenticated and granted access.
Basic Flow	1. User enters credentials. 2. System authenticates against stored hash. 3. Access granted, session created.
Includes	Authenticate Credentials (internal)

Table 3-20: Use Case UC03 – Reset Password

Field	Description
Use Case ID	UC03
Use Case Name	Reset Password
Actor	Guest
Precondition	Registered email address exists.
Postcondition	User can log in with new password.
Basic Flow	1. User clicks “Forgot Password”. 2. User enters email. 3. System sends password reset link.

	4. User clicks link, enters new password. 5. System updates password.

Table 3-21: Use Case UC04 – Logout

Field	**Description**
Use Case ID	UC04
Use Case Name	Logout
Actor	User
Precondition	User is logged in.
Postcondition	Session terminated.
Basic Flow	1. User taps Logout. 2. System clears local session and tokens.

Table 3-22: Use Case UC05 – Update Profile

Field	**Description**
Use Case ID	UC05
Use Case Name	Update Profile
Actor	User
Precondition	User is logged in.
Postcondition	Profile information (name, email) updated.
Basic Flow	1. User edits profile fields. 2. System saves changes.

Expense Management

Table 3-23: Use Case UC06 – Set Monthly Budget

Field	**Description**
Use Case ID	UC06
Use Case Name	Set Monthly Budget
Actor	User
Precondition	User logged in.
Postcondition	Monthly budget limit stored.
Basic Flow	1. User enters budget amount. 2. System saves budget for the current month.

Table 3-24: Use Case UC07 – Add Expense

Field	**Description**
Use Case ID	UC07
Use Case Name	Add Expense
Actor	User
Precondition	User authenticated.
Postcondition	Expense stored locally (and later synced).
Basic Flow	1. User enters amount, category, date, description. 2. System validates input. 3. System saves expense.
Includes	Validate Input
Extends	Suggest Personalized Categories (if applicable)

Table 3-25: Use Case UC08 – Edit Expense

Field	**Description**
Use Case ID	UC08
Use Case Name	Edit Expense
Actor	User
Precondition	Expense exists.
Postcondition	Expense updated.
Basic Flow	1. User selects an expense. 2. User modifies fields. 3. System saves changes.

Table 3-26: Use Case UC09 – Delete Expense

Field	**Description**
Use Case ID	UC09
Use Case Name	Delete Expense
Actor	User
Precondition	Expense exists.
Postcondition	Expense removed from storage.
Basic Flow	1. User selects an expense. 2. System asks for confirmation. 3. User confirms, system deletes.

Table 3-27: Use Case UC10 – View Expenses

Field	Description
Use Case ID	UC10
Use Case Name	View Expenses
Actor	User
Precondition	User logged in.
Postcondition	List of expenses displayed.
Basic Flow	1. System retrieves expenses. 2. System displays list sorted by date.

Table 3-28: Use Case UC11 – Filter Expenses

Field	Description
Use Case ID	UC11
Use Case Name	Filter Expenses
Actor	User
Precondition	Expenses exist.
Postcondition	Filtered list displayed.
Basic Flow	1. User selects date range and/or category. 2. System displays matching expenses.

Table 3-29: Use Case UC12 – Search Expenses by Keyword

Field	Description
Use Case ID	UC12
Use Case Name	Search Expenses by Keyword
Actor	User
Precondition	Expenses exist.
Postcondition	Matching expenses displayed.
Basic Flow	1. User enters keyword (partial match). 2. System searches description field (SQL LIKE). 3. Results displayed.

Debt Management

Table 3-30: Use Case UC13 – Record Borrowed Money

Field	**Description**
Use Case ID	UC13
Use Case Name	Record Borrowed Money
Actor	User
Precondition	User logged in.
Postcondition	Debt record stored.
Basic Flow	1. User enters person's name, amount, date, due date (optional). 2. System saves as "I owe" debt.

Table 3-31: Use Case UC14 – Record Lent Money

Field	**Description**
Use Case ID	UC14
Use Case Name	Record Lent Money
Actor	User
Precondition	User logged in.
Postcondition	Debt record stored.
Basic Flow	1. User enters person's name, amount, date, due date (optional). 2. System saves as "Others owe me" debt.

Table 3-32: Use Case UC15 – Settle Debt

Field	**Description**
Use Case ID	UC15
Use Case Name	Settle Debt
Actor	User
Precondition	Debt exists and is pending.
Postcondition	Debt marked as fully settled.
Basic Flow	1. User selects a debt. 2. User marks as settled. 3. System updates status.
Extends	Send Notification (optional, to remind or confirm)

Table 3-33: Use Case UC16 – Partial Repayment

Field	**Description**
Use Case ID	UC16
Use Case Name	Partial Repayment
Actor	User
Precondition	Debt exists and is pending.
Postcondition	Remaining balance updated.
Basic Flow	1. User selects a debt. 2. User enters repayment amount. 3. System subtracts from remaining balance.

Table 3-34: Use Case UC17 – View Debts

Field	**Description**
Use Case ID	UC17
Use Case Name	View Debts
Actor	User
Precondition	User logged in.
Postcondition	Two lists displayed: “I owe” and “Others owe me”.
Basic Flow	1. System retrieves all debts. 2. System separates into borrowed and lent lists. 3. Displays with balances.

Table 3-35: Use Case UC18 – Search Debts by Name

Field	**Description**
Use Case ID	UC18
Use Case Name	Search Debts by Name
Actor	User
Precondition	Debt records exist.
Postcondition	Matching debts displayed.
Basic Flow	1. User enters person’s name (partial). 2. System shows all debts involving that person.

Table 3-36: Use Case UC19 – Edit Debt

Field	**Description**
Use Case ID	UC19
Use Case Name	Edit Debt
Actor	User
Precondition	Debt exists.
Postcondition	Debt details updated.
Basic Flow	1. User selects a debt. 2. User modifies amount, due date, etc. 3. System saves changes.

Table 3-37: Use Case UC20 – Delete Debt

Field	**Description**
Use Case ID	UC20
Use Case Name	Delete Debt
Actor	User
Precondition	Debt exists.
Postcondition	Debt removed.
Basic Flow	1. User selects a debt. 2. Confirmation prompts. 3. System deletes record.

Insights & Visualization

Table 3-38: Use Case UC21 – View Pie Charts by Category

Field	**Description**
Use Case ID	UC21
Use Case Name	View Pie Charts by Category
Actor	User
Precondition	Expense data exists for selected date range.
Postcondition	Pie chart displayed.
Basic Flow	1. User selects date range (week, month, custom). 2. System calculates percentage per category. 3. System renders pie chart.

Table 3-39: Use Case UC22 – View Spending Trends

Field	**Description**
Use Case ID	UC22
Use Case Name	View Spending Trends
Actor	User
Precondition	Expense data exists.
Postcondition	Line/bar chart of daily/weekly spending displayed.
Basic Flow	1. User selects time period. 2. System aggregates spending per day/week. 3. System renders trend graph.

Table 3-40: Use Case UC23 – Compare Spending vs Budget

Field	**Description**
Use Case ID	UC23
Use Case Name	Compare Spending vs Budget
Actor	User
Precondition	Monthly budget set.
Postcondition	Comparison (spent vs budget) displayed.
Basic Flow	1. System calculates month-to-date spending. 2. System compares with budget. 3. Displays percentage used and remaining.

Table 3-41: Use Case UC24 – View Highest Spending Category

Field	**Description**
Use Case ID	UC24
Use Case Name	View Highest Spending Category
Actor	User
Precondition	Expense data exists.
Postcondition	Category with largest total shown, with percentage.
Basic Flow	1. System sums expenses per category. 2. Sorts descending. 3. Displays top category and its share.

Integrated Dashboard

Table 3-42: UC41 – View Dashboard

Field	Description
Use Case ID	UC41
Use Case Name	View Dashboard
Actor	User
Precondition	User logged in.
Postcondition	Dashboard summary displayed.
Basic Flow	1. System shows: - Spending vs budget (UC23) - Highest spending category (UC24) - Budget prediction (UC28) - Smart recommendation (UC27, if any) - Recent expenses list
Includes	UC23, UC24, UC28
Extends	UC27 (optional)

Notifications & Reminders

Table 3-43: UC30 – Set Manual Reminder

Field	Description
Use Case ID	UC30
Use Case Name	Set Manual Reminder
Actor	User
Precondition	User logged in.
Postcondition	Reminder scheduled.
Basic Flow	1. User enters title, date/time, recurrence (optional). 2. System stores reminder.

Table 3-44: UC31 – Send Online Notification

Field	Description
Use Case ID	UC31
Use Case Name	Send Online Notification
Actor	Automation
Precondition	Device has internet connection.

Postcondition	Notification delivered.
Basic Flow	1. System triggers a notification (due debt, budget alert, etc.). 2. System sends push notification immediately.

Table 3-45: UC32 – Queue Offline Notification

Field	**Description**
Use Case ID	UC32
Use Case Name	Queue Offline Notification
Actor	Automation
Precondition	Device is offline.
Postcondition	Notification stored locally.
Basic Flow	1. System generates a notification. 2. Detects no internet. 3. Stores notification in local queue. 4. When connectivity returns (UC35), sends queued notifications.

Offline Support

Table 3-46: UC33 – Store Data Locally (SQLite)

Field	**Description**
Use Case ID	UC33
Use Case Name	Store Data Locally
Actor	(internal)
Precondition	App is installed.
Postcondition	All user data persists in SQLite.
Basic Flow	1. Every create/update/delete operation writes to local SQLite database.

Table 3-47: UC34 – Operate Fully Offline

Field	**Description**
Use Case ID	UC34
Use Case Name	Operate Fully Offline
Actor	(capability)
Precondition	No internet connection.
Postcondition	All features accessible without network.

Basic Flow	1. User adds, edits, deletes expenses/debts. 2. All changes stored locally. 3. No blocking for lack of internet.

Table 3-48: UC35 – Sync Data when Online

Field	**Description**
Use Case ID	UC35
Use Case Name	Sync Data when Online
Actor	System
Precondition	Internet connection restored.
Postcondition	Local and server data are Synchronized.
Basic Flow	1. System uploads local changes with “unsynced” flag. 2. System downloads remote changes since last synchronization. 3. The System Resolves conflicts. 4. Updates local synchronization timestamp.

Data Backup & Recovery

Table 3-49: UC36 – Manual Backup

Field	**Description**
Use Case ID	UC36
Use Case Name	Manual Backup
Actor	User
Precondition	User logged in.
Postcondition	Backup file created locally.
Basic Flow	1. User selects “Backup now”. 2. System exports all expenses, debts, reminders to a local file.

Table 3-50: UC37 – Auto Backup (daily)

Field	**Description**
Use Case ID	UC37
Use Case Name	Auto Backup (daily)
Actor	Automation
Precondition	App is idle or closed.
Postcondition	Daily backup saved.
Basic Flow	1. System creates backup automatically without user intervention.

Table 3-51: UC38 – Restore Data

Field	**Description**
Use Case ID	UC38
Use Case Name	Restore Data
Actor	User
Precondition	A backup file exists (local or server).
Postcondition	All data restored.
Basic Flow	1. User re-installs app or selects “Restore”. 2. System retrieves latest backup. 3. Overwrites current data with backup.

Category Customization

Table 3-52: UC39 – Create Personal Category

Field	**Description**
Use Case ID	UC39
Use Case Name	Create Personal Category
Actor	User
Precondition	User logged in.
Postcondition	New category added to user’s list.
Basic Flow	1. User enters category name and optional icon. 2. System saves custom category alongside predefined ones.

Table 3-53: UC40 – Edit/Delete Personal Category

Field	**Description**
Use Case ID	UC40
Use Case Name	Edit/Delete Personal Category
Actor	User
Precondition	User has created personal categories.
Postcondition	Category updated or removed.
Basic Flow	1. User selects a personal category. 2. User edits name or deletes. 3. System saves changes. (Deletion moves existing expenses to “Other” category.)

3.4.2 Sequence Diagram

This subsection presents sequence diagrams for the most critical interactions in the Adaptive Budgeting and Expense Tracker. Each diagram illustrates the flow of messages between actors and system components over time, highlighting offline-first behaviour, rule-based adaptive features, and synchronisation.

Add Expense Sequence Diagram

This diagram illustrates the process of recording a new expense. The mobile application validates the input and stores the expense locally in SQLite. When internet connectivity is available, the expense is immediately synchronized with the backend API; otherwise, it is marked as pending synchronization and uploaded later when connectivity is restored. After storage, the system performs adaptive checks such as budget threshold analysis and unusual spending detection before displaying relevant alerts.

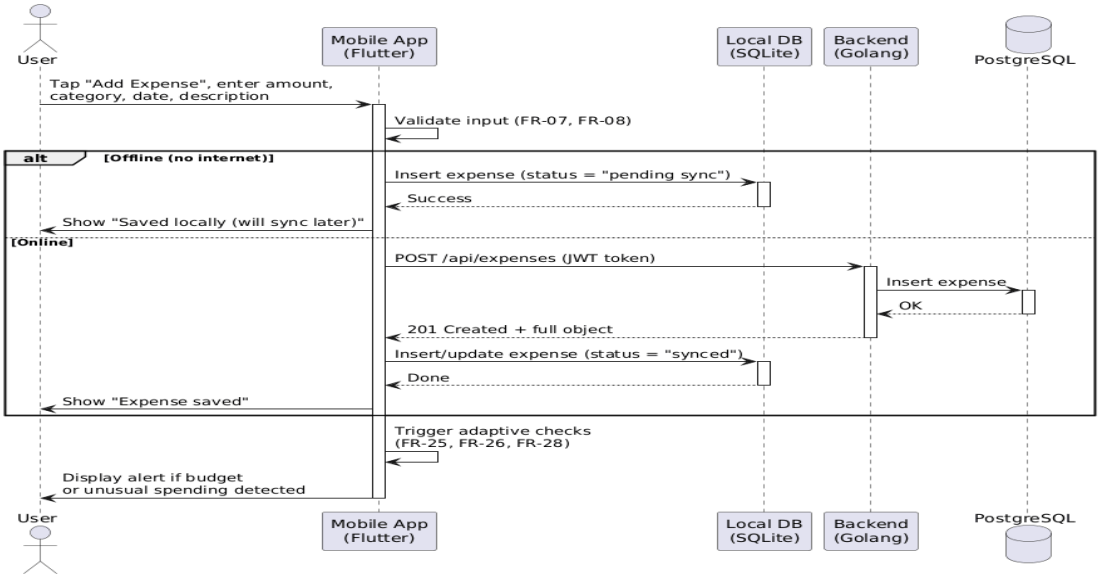


Figure 3-2: Add Expense Sequence Diagram

Debt Recording Sequence Diagram

This diagram illustrates the process of recording money lent to another person. The application stores the debt locally while distinguishing between borrowed and lent transactions. If the device is online, the debt information is synchronized immediately with the backend server; otherwise, synchronization is deferred until connectivity becomes available. The system also evaluates due dates and generates reminder notifications when necessary.

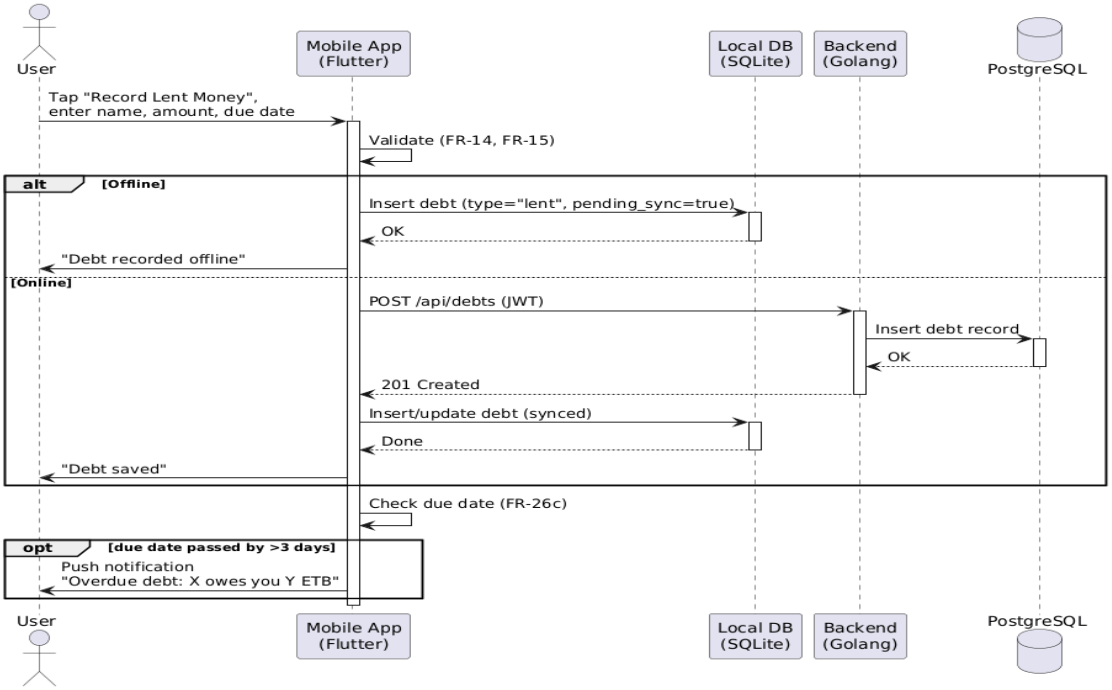


Figure 3-3: Debt Recording Sequence Diagram

Background Synchronization Sequence Diagram

This diagram illustrates the automatic synchronization process triggered when internet connectivity becomes available. The application retrieves remote updates from the server, merges them with local records using a server-wins conflict resolution strategy, and uploads locally pending records. The synchronization process operates automatically in the background without interrupting user activities.

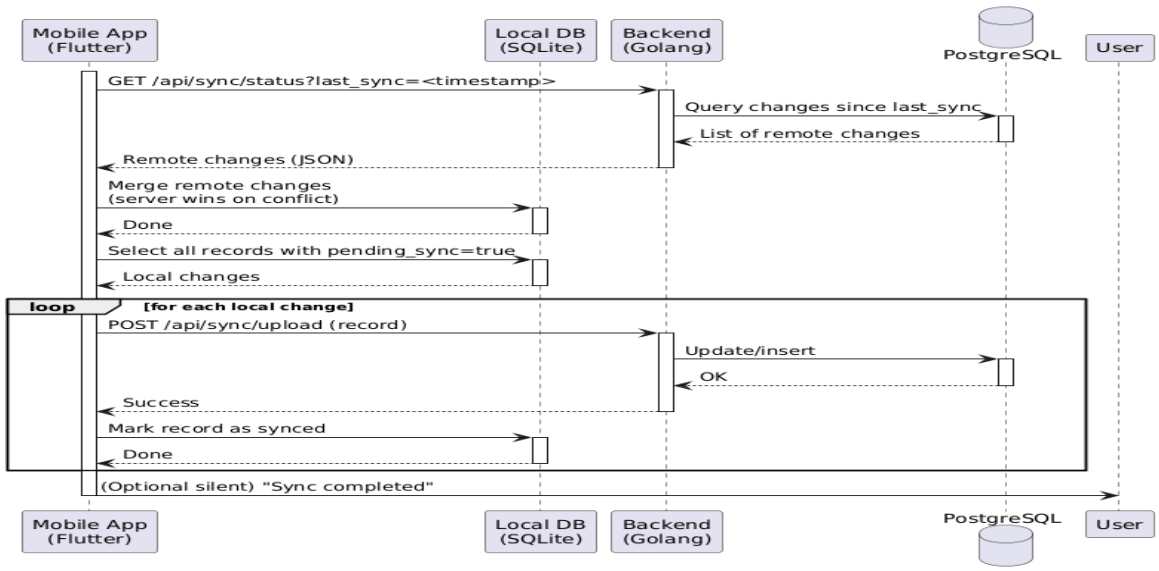


Figure 3-4: Background Synchronization Sequence Diagram

Dashboard Analytics and Prediction Sequence Diagram

This diagram illustrates the generation of dashboard analytics and adaptive predictions. When the user accesses the dashboard, the application retrieves expense records from the local database, calculates category-based spending totals, compares expenses against the monthly budget, identifies the highest spending category, and predicts end-of-month spending using a moving average technique. The system then generates contextual recommendations and displays summarized financial insights.

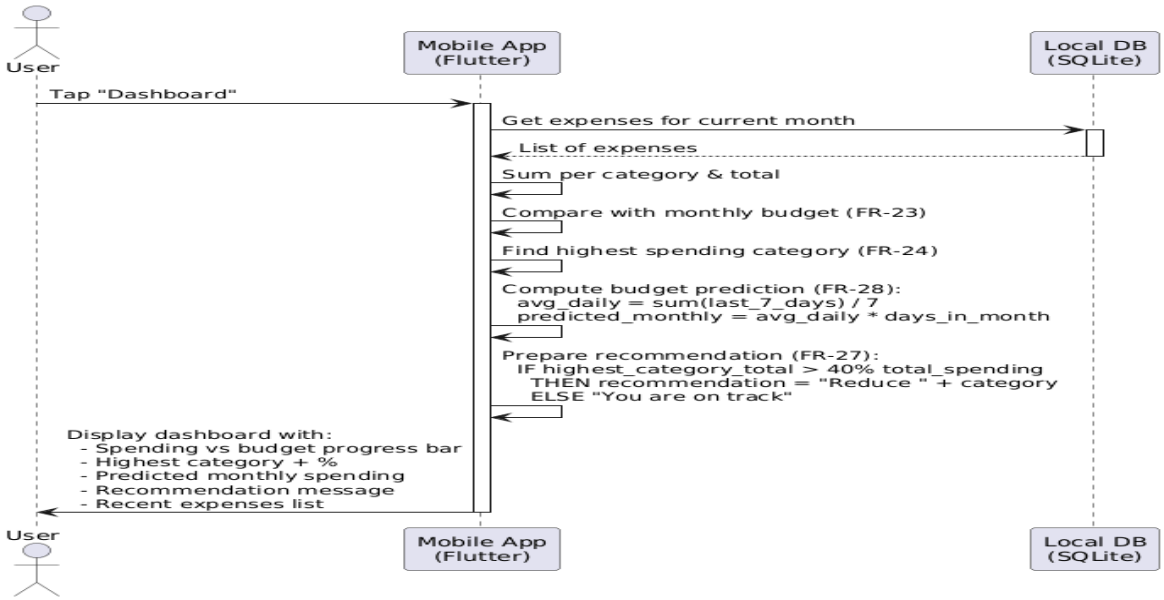


Figure 3-5: Dashboard Analytics and Prediction Sequence Diagram

Reminder Detection Sequence Diagram

This diagram illustrates the detection of recurring expense patterns for reminder generation. After a new expense is recorded, the system analyses previous transactions to identify repeated spending patterns based on similar amounts and recurring time intervals. When a recurring pattern is detected, the application prompts the user to create a reminder for future occurrences.

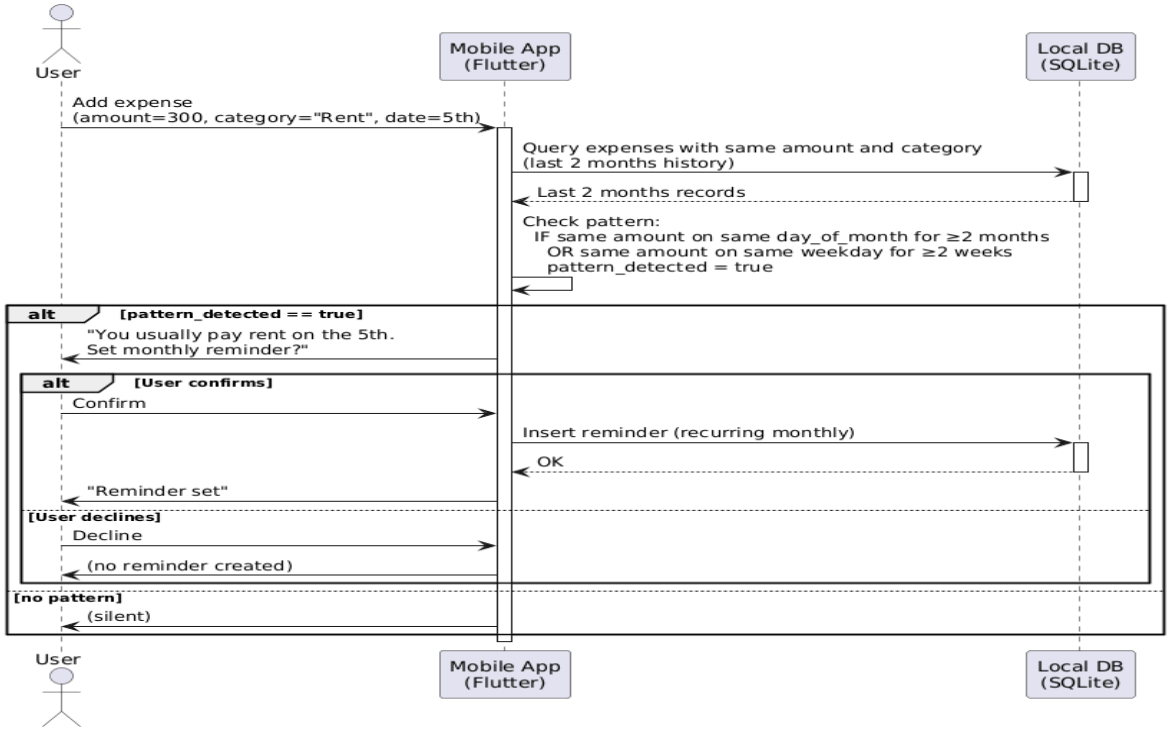


Figure 3-6: Reminder Detection Sequence Diagram

Password Reset Sequence Diagram

A user who forgot their password can request a reset link. The backend generates a secure one-time token, stores its hash, and sends an email with a reset link. When the user clicks the link and provides a new password, the backend validates the token and updates the password after hashing. Generic error messages prevent email enumeration.

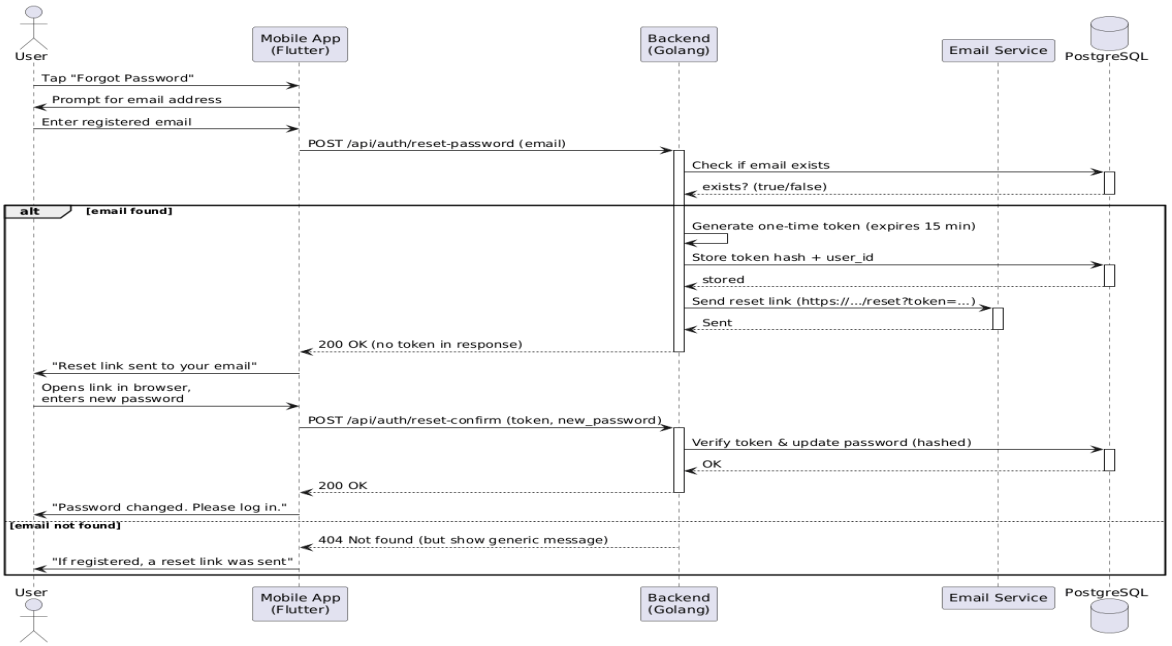


Figure 3-7: Password Reset Sequence Diagram

3.4.3 Activity Diagram

This subsection presents activity diagrams for the key workflows of the Adaptive Budgeting and Expense Tracker. Activity diagrams model the flow of control from one activity to another, showing decision points, parallel execution, and overall process logic.

Add Expense Workflow

This activity diagram illustrates the complete process of adding a new expense, including input validation, offline-first storage, synchronisation decision, and post-saving adaptive checks (budget alert, unusual spending detection). The diagram shows how the system behaves both online and offline.

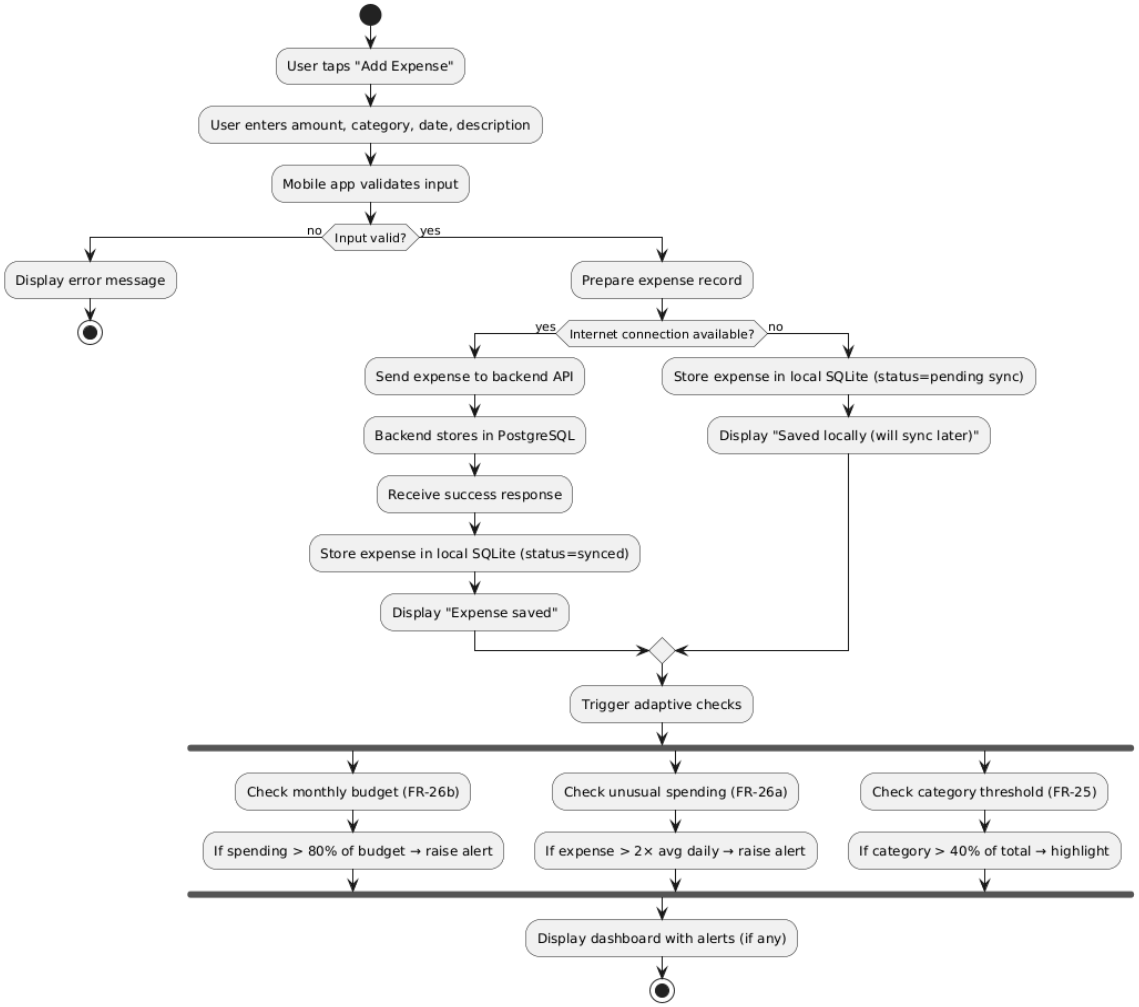
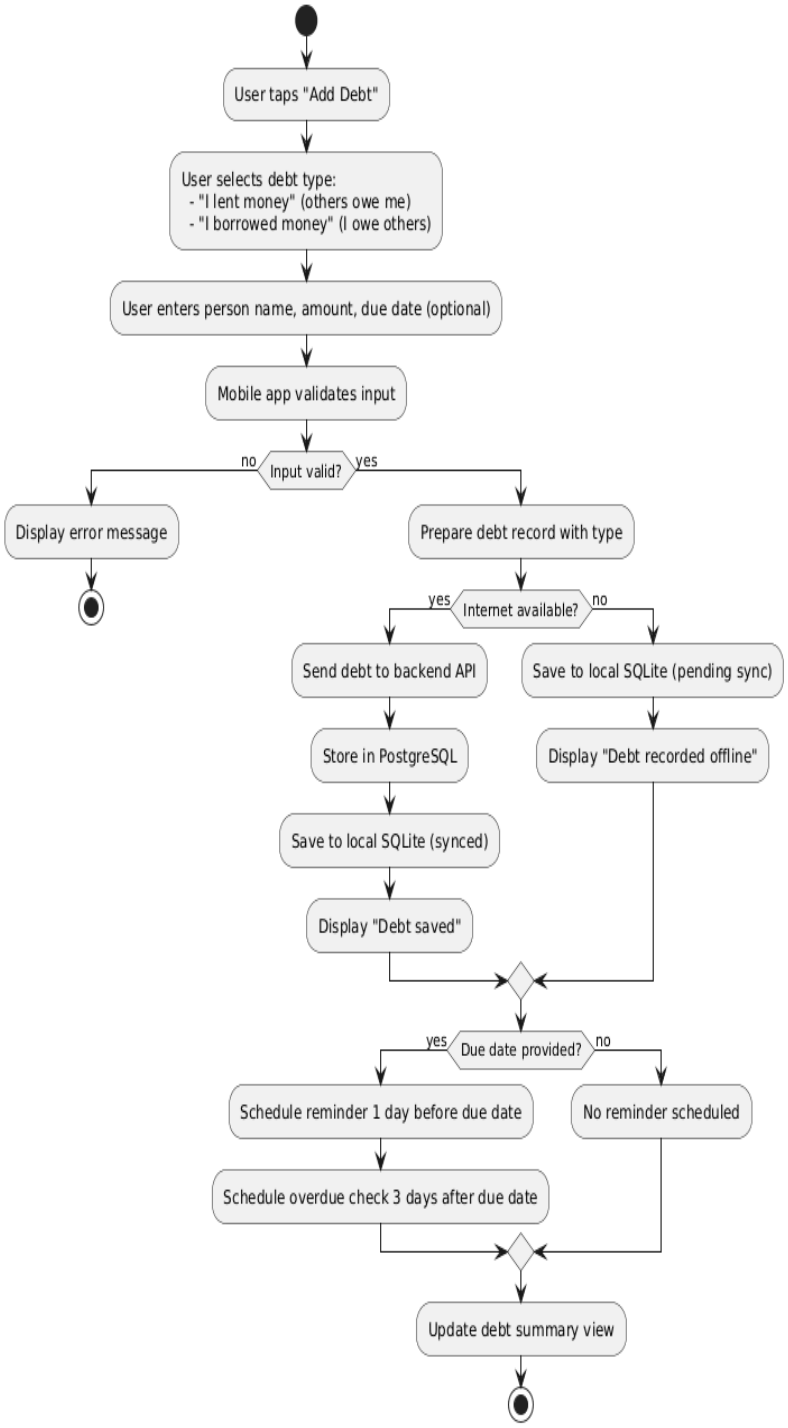


Figure 3-8: Add Expense Workflow (Activity Diagram)

Debt Recording Workflow

This diagram shows the workflow for recording a peer-to-peer debt. The user selects whether they lent money or borrowed money, enters the counterparty name, amount, and optional due

date. The system stores the debt offline-first and optionally schedules a reminder notification based on the due date.



3-9: Debt Recording Workflow (Activity Diagram)

Background Synchronization Workflow

This diagram models the background synchronisation process that runs when the device regains internet connectivity. It pulls remote changes from the server, merges them into the

local database (server wins on conflicts), and pushes any local pending records. The process is non-blocking and runs automatically.

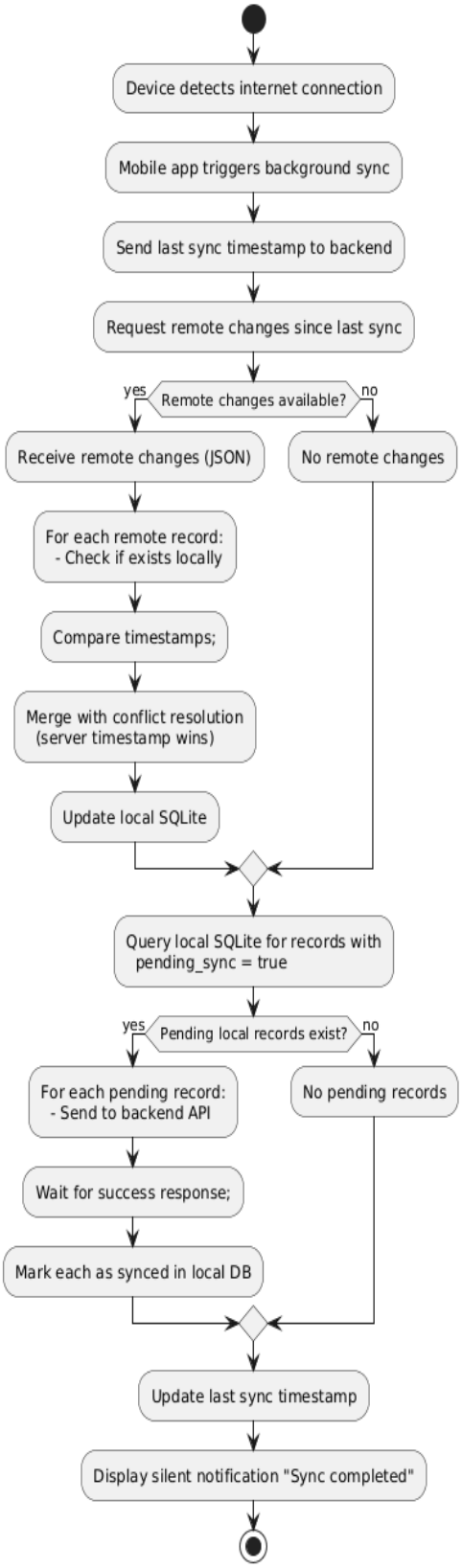


Figure 3-10: Background Synchronization Workflow (Activity Diagram)

Dashboard Analytics and Prediction Workflow

When the user opens the dashboard, the system retrieves current month expenses, calculates spending totals by category, compares with the monthly budget, identifies the highest spending category, and generates a budget prediction using a moving average. All results are displayed in a single summary screen.

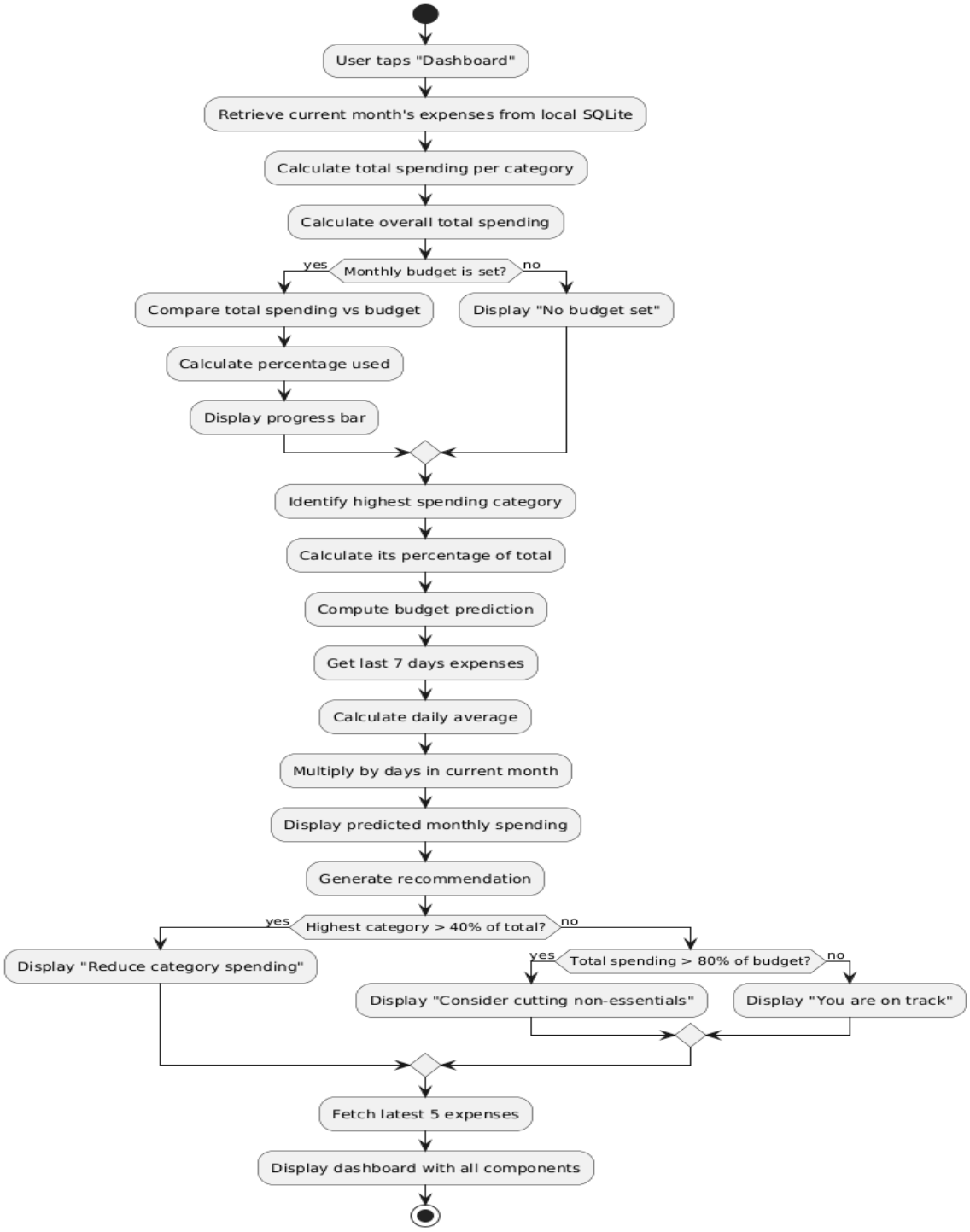


Figure 3-11: Dashboard Analytics and Prediction Workflow (Activity Diagram)

Password Reset Workflow

This activity diagram models the complete password recovery flow. The user requests a reset link, receives an email, clicks the link, and sets a new password. The diagram includes error handling for invalid tokens and email not found (using generic messages for security).

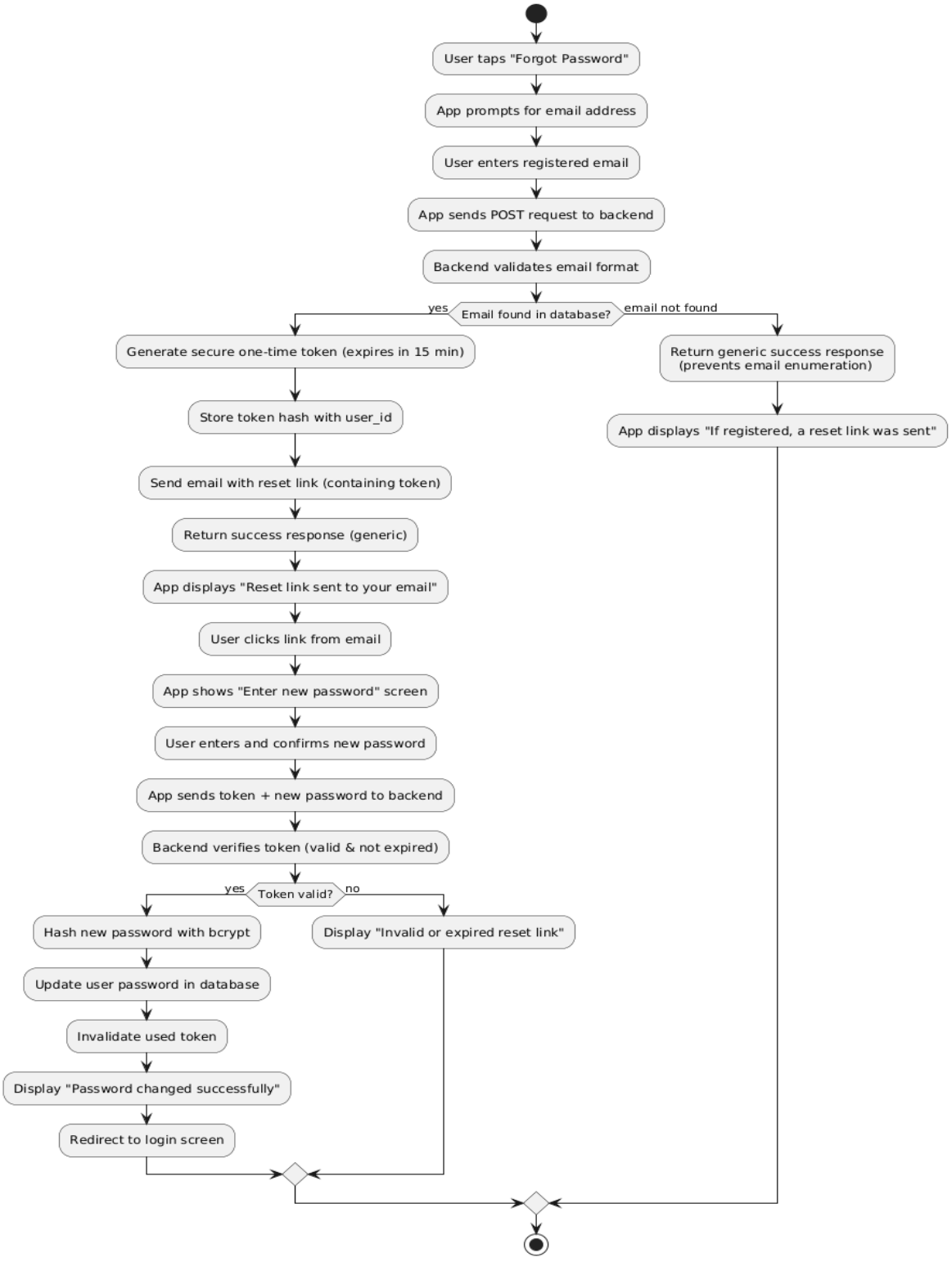


Figure 3-12: Password Reset Workflow (Activity Diagram)

Reminder Detection Workflow

After a user adds an expense, this workflow detects recurring patterns based on amount and time interval (same day of month for two months, or same weekday for two weeks). If a pattern is found, the user is prompted to set an automated reminder.

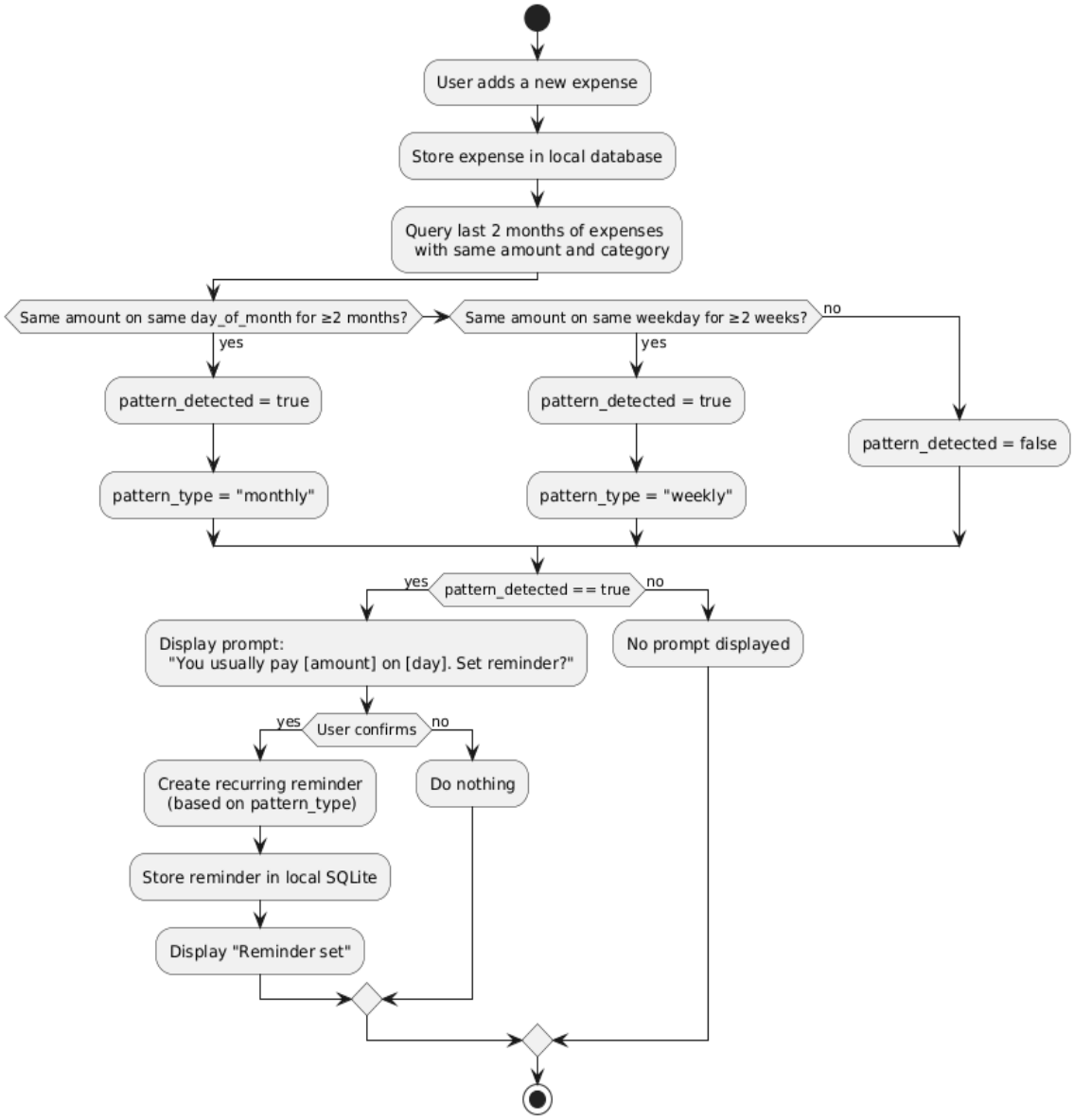


Figure 3-13: Reminder Detection Workflow (Activity Diagram)

3.4.4 Class Diagram

The class diagram defines the static structure of the Adaptive Budgeting and Expense Tracker. It presents the principal classes (entities), their attributes, methods, and the relationships between them. This diagram serves as a blueprint for both the backend database (PostgreSQL) and the local mobile database (SQLite), ensuring consistency between offline and online storage. The design follows object-oriented principles (encapsulation, inheritance

where applicable) and directly supports the functional requirements and use cases defined earlier in this chapter.

Overview of Main Classes

The system is organised around six core classes:

Table 3-54: Overview of Main Classes

Class	Purpose
User	Stores account information, credentials, and user-specific settings (e.g., monthly budget).
Category	Manages expense categories – both predefined (e.g., Food, Transport) and user-created.
Expense	Represents a single cash transaction with amount, date, description, and a link to a category.
Debt	Records peer-to-peer informal debts: money lent to someone else or money borrowed from someone.
Reminder	Handles both manual and automatically triggered reminders (recurring expenses, debt due dates).
SyncLog	Tracks synchronisation events between the mobile device and the backend server (offline-first support).

Class Descriptions

Class User

The User class represents a registered individual. It holds authentication data, profile information, and the user's optional monthly budget limit. The class is responsible for managing the user's lifecycle: registration, login, password reset, profile updates, and retrieving associated expenses/debts.

Table 3-55: Class User – Attributes

Attribute	Type	Description	Constraints / Notes
userId	int	Primary key, unique identifier	Auto-increment, not null
email	String	Login credential and contact	Unique, valid email format
passwordHash	String	bcrypt hash of the user's password	Stored, never plain text
fullName	String	User's display name	Optional, max 100 chars

monthlyBudget	double	Monthly spending limit	Default = 0 (not set)
createdAt	Date	Account creation timestamp	Auto-set on registration
lastSyncTimestamp	Date	Last successful sync with server	Updated after each sync

Table 3-56: Class User – Methods

Method	**Description**	**Related Use Case / FR**
register()	Creates a new user account after validating email and password strength.	UC01, FR-01, FR-02, FR-03
login()	Authenticates user credentials and creates a session.	UC02, NFR-08
resetPassword()	Initiates password reset flow (sends email token).	UC03, FR-38, FR-39
updateProfile()	Updates fullName or other editable fields.	UC05, FR-06
setMonthlyBudget()	Sets or changes the monthlyBudget attribute.	UC06, FR-07
getExpenses()	Returns a list of all expenses belonging to this user.	UC10
getDebts()	Returns a list of all debts belonging to this user.	UC17

Relationships

- One User can have zero or many Expense objects (composition).
- One User can have zero or many Debt objects (composition).
- One User can have zero or many Reminder objects (composition).
- One User can have zero or many SyncLog entries (composition).

Class Category

The Category class classifies expenses. The system provides a set of predefined categories (e.g., Food, Transport, Education, Entertainment). Additionally, users can create their own personal categories (FR-43) and edit or delete them (FR-44). Predefined categories are distinguished by isPredefined = true and userId = null.

Table 3-57: Class Category – Attributes

Attribute	**Type**	**Description**	**Constraints**
categoryId	int	Primary key	Auto-increment
userId	int	Foreign key to User (nullable)	Null for predefined categories
name	String	Display name of the category	Max 30 chars, unique per user + predefined

icon	String	Emoji or icon identifier (e.g., "🍔")	Optional
isPredefined	boolean	True if system-defined, false if user-created	Default = false

Table 3-58: Class Category – Methods

Method	**Description**	**Related FR / UC**
createCategory()	Creates a new personal category for a user.	UC39, FR-43
editCategory()	Renames a user-created category.	UC40, FR-44
deleteCategory()	Deletes a personal category; reassigns its expenses to "Other".	UC40, FR-44
getExpenses()	Returns all expenses that use this category.	(Helper)

Relationships

- One Category can be associated with zero or many Expense objects.
- Predefined categories are shared across all users (no userId).
- Personal categories belong to exactly one user (userId not null).

Class Expense

The Expense class records a single cash transaction. To support offline-first operation, each expense has a syncStatus indicating whether it has been synchronised with the backend. Expenses are stored both locally (SQLite) and remotely (PostgreSQL). The class also enables editing, deletion, filtering, and adaptive analysis.

Table 3-59: Class Expense – Attributes

Attribute	**Type**	**Description**	**Constraints**
expenseId	int	Primary key	Auto-increment
userId	int	Foreign key to User	Not null
categoryId	int	Foreign key to Category	Not null
amount	double	Expense amount	> 0
expenseDate	Date	Date when the transaction occurred	Default = current date
description	String	Optional note about the expense	Max 200 chars
isSynced	boolean	True if already uploaded to server	Default = false
syncStatus	String	"pending" or "synced"	Used for conflict resolution

Table 3-60: Class Expense – Methods

Method	Description	Related FR / UC
addExpense()	Inserts a new expense record, triggers adaptive checks.	UC07, FR-08, FR-09
editExpense()	Modifies an existing expense (amount, date, category).	UC08, FR-10
deleteExpense()	Removes an expense after confirmation.	UC09, FR-11
getCategory()	Returns the associated Category object.	(Helper)

Relationships

- Many Expense objects belong to one User.
- Many Expense objects belong to one Category.

Offline behaviour – When created offline, isSynced = false and syncStatus = "pending". After a successful sync, both flags are updated.

Class Debt

The Debt class models informal peer-to-peer lending and borrowing. A single boolean flag isLent distinguishes between “I lent money to someone” (others owe me) and “I borrowed money from someone” (I owe others). Support for partial repayment is provided via remainingAmount. The class also tracks due dates and sends reminders when a debt becomes overdue (FR-26).

Table 3-61: Class Debt – Attributes

Attribute	Type	Description	Constraints
debtId	int	Primary key	Auto-increment
userId	int	Foreign key to User	Not null
counterpartyName	String	Name of the other person	Max 100 chars
amount	double	Original debt amount	> 0
isLent	boolean	True = others owe me, False = I owe others	Not null
remainingAmount	double	Current outstanding balance	≥ 0, ≤ amount
dueDate	Date	Optional date by which debt should be settled	Can be null
status	String	“pending” or “settled”	Default = “pending”

notes	String	Additional remarks	Optional
isSynced	boolean	Same as in Expense	Default = false

Table 3-62: Class Debt – Methods

Method	**Description**	**Related FR / UC**
recordDebt()	Creates a new debt (lent or borrowed).	UC13, UC14, FR-14, FR-15
settleDebt()	Marks debt as fully settled; updates status and remainingAmount.	UC15, FR-17
partialRepayment()	Reduces remainingAmount by a given amount.	UC16, FR-18
editDebt()	Changes counterparty, amount, due date, etc.	UC19, FR-19
deleteDebt()	Removes the debt record.	UC20, FR-19

Relationships

- Many Debt objects belong to one User.

Key logic:

- If isLent = true, the user is expecting repayment; remainingAmount decreases when the counterparty pays back.
- If isLent = false, the user owes money; remainingAmount decreases when the user repays.
- When remainingAmount becomes 0, status is set to "settled".

Class Reminder

The Reminder class handles push notifications (FR-32, FR-33). Reminders can be created manually by the user or generated automatically by the smart reminder detection (FR-31). A reminder may be associated with an Expense (for recurring spending) or a Debt (for due dates). Recurrence patterns (daily, weekly, monthly) are supported.

Table 3-63: Class Reminder – Attributes

Attribute	**Type**	**Description**	**Constraints**
reminderId	int	Primary key	Auto-increment
userId	int	Foreign key to User	Not null
expenseId	int	Optional reference to an Expense	Nullable
debtId	int	Optional reference to a Debt	Nullable
title	String	Short description (e.g., "Pay rent")	Max 100 chars
reminderDateTime	Date	When to trigger the notification	Not null

recurrencePattern	String	“none”, “daily”, “weekly”, “monthly”	Default = “none”
isActive	boolean	Whether the reminder is enabled	Default = true
lastTriggered	Date	Last time notification was sent	Auto-updated

Table 3-64: Class Reminder – Methods

Method	**Description**	**Related FR / UC**
setReminder()	Creates a new manual reminder.	UC30, FR-32
editReminder()	Modifies title, date/time, recurrence.	–
deleteReminder()	Permanently removes the reminder.	–
triggerNotification()	Sends push notification (or queues if offline).	UC31, UC32, FR-33, FR-34

Relationships

- Many Reminder objects belong to one User.
- Optionally, a Reminder may reference zero or one Expense (if tied to a recurring expense).
- Optionally, a Reminder may reference zero or one Debt (if tied to a debt due date).

Smart reminder detection – After adding an expense, the system queries for patterns (same amount, same day of month). If detected, it calls setReminder() with a suggested recurrence pattern (FR-31).

Class SyncLog

The SyncLog class maintains a history of synchronisation operations. It is used to resolve conflicts (server timestamp wins) and to determine what data needs to be uploaded or downloaded. Each sync event is recorded for debugging and transparency.

Table 3-65: Class SyncLog – Attributes

Attribute	**Type**	**Description**
syncId	int	Primary key, auto-increment
userId	int	Foreign key to User
lastSyncAt	Date	Timestamp when this sync completed
recordsUploaded	int	Number of pending records sent to server
recordsDownloaded	int	Number of remote changes pulled
status	String	“success”, “partial”, or “failed”

Table 3-66: Class SyncLog – Methods

Method	**Description**

logSync()	Inserts a new sync record after each synchronisation attempt.
getLastSync()	Returns the most recent lastSyncAt for a given user.

Relationships

- Many SyncLog entries belong to one User.

Usage – Before synchronising, the app calls getLastSync() to obtain the timestamp. Changes after that timestamp are pulled from the server. After a successful sync, a new SyncLog entry is created.

Relationships Summary (Cardinalities)

The diagram uses standard UML notation:

- 1 – exactly one
- * – zero or many
- 0..1 – optional one

Table 3-67: Relationships Summary (Cardinalities)

From	**To**	**Cardinality**	**Type**
User	Expense	1 → *	Composition
User	Debt	1 → *	Composition
User	Reminder	1 → *	Composition
User	SyncLog	1 → *	Composition
Category	Expense	1 → *	Aggregation (shared)
Reminder	Expense	0..1 → 0..1	Optional reference
Reminder	Debt	0..1 → 0..1	Optional reference

Note: The relationship between Reminder and Expense/Debt is not a composition; a reminder can exist without being linked to any expense or debt (manual reminders). Conversely, an expense can have multiple reminders (e.g., a weekly reminder for the same expense). The diagram shows a simple optional reference.

Mapping to Functional Requirements

The following table links each class to the key requirements it fulfils:

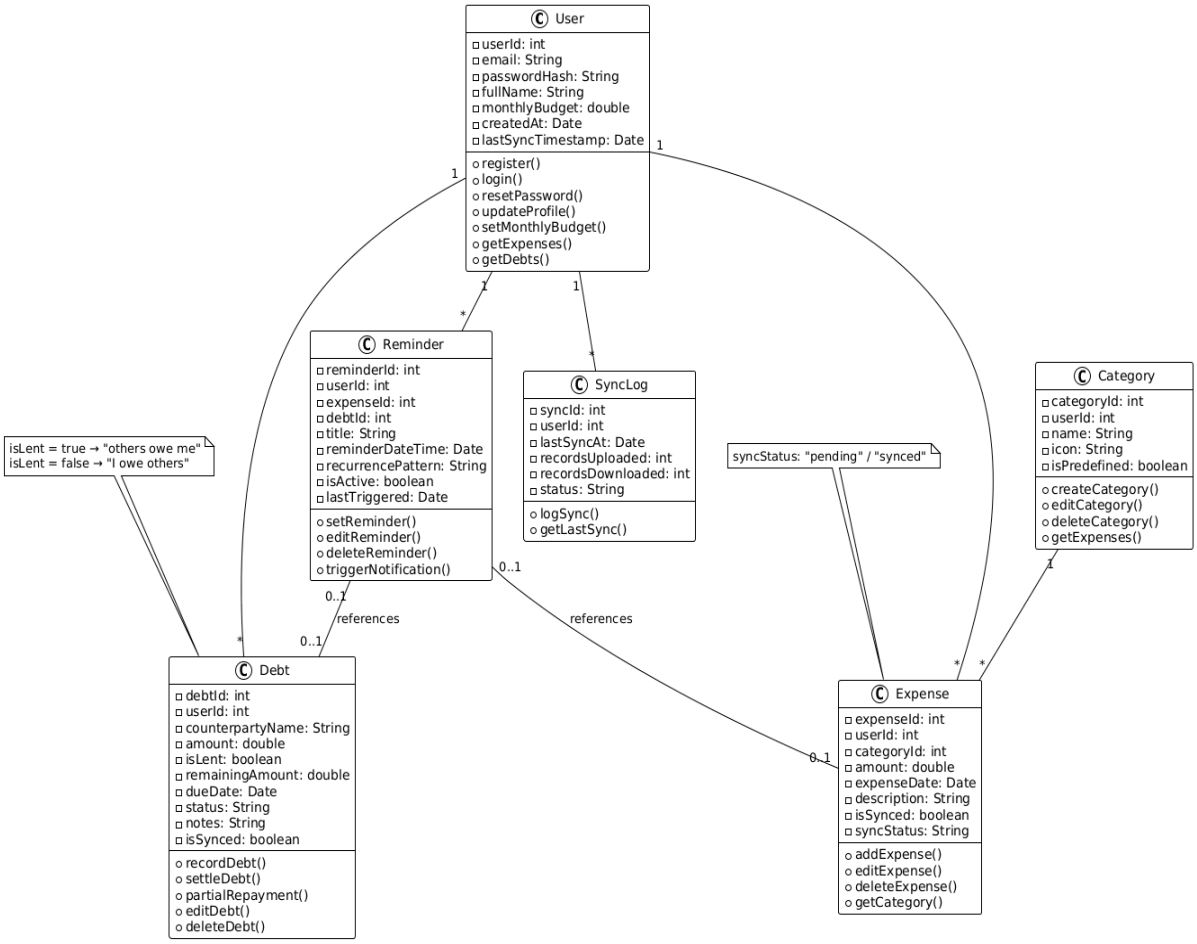
Table 3-68: Mapping of Classes to Functional Requirements

Class	**Fulfilled Requirements**
User	FR-01 to FR-07, NFR-08, NFR-09
Category	FR-09, FR-43, FR-44
Expense	FR-08 to FR-13, FR-25, FR-26, FR-28
Debt	FR-14 to FR-20, FR-26

Reminder	FR-31 to FR-34
SyncLog	FR-35 to FR-37

Implementation Mapping (Database & Code)

- PostgreSQL (Backend) – Each class maps to a table with columns matching the attributes. Foreign keys enforce referential integrity.
- SQLite (Mobile local DB) – Identical schema to enable offline operations and easy synchronisation.
- Golang (Backend) – Structs with GORM annotations for ORM.
- Flutter/Dart (Mobile) – Data models with toJson()/fromJson() methods for serialisation; floor or sqflite for local storage.



3-14: Class Diagram of the Adaptive Budgeting and Expense Tracker

Chapter Four
System Design

4.1 Introduction

System design is a critical phase in software development that transforms the requirements and analysis models identified during system analysis into a detailed technical blueprint for implementation. The purpose of this chapter is to present the overall design of the Adaptive Budgeting and Expense Tracker and describe how the proposed system will be structured, organized, and implemented.

The design focuses on ensuring that the system satisfies the functional and non-functional requirements defined in Chapter Three while maintaining simplicity, scalability, maintainability, reliability, and usability. Emphasis is given to offline-first operation, secure data management, synchronization between local and remote storage, and support for adaptive rule-based financial features.

The proposed system is designed as a cross-platform mobile application using Flutter for the frontend, Golang for the backend REST API, SQLite for local mobile storage, and PostgreSQL for centralized server-side data storage. The system architecture supports both online and offline operation, enabling users to continue recording expenses and debts even without internet connectivity.

4.2 System Architecture Design

Overview of Architecture

The Adaptive Budgeting and Expense Tracker follow a three-tier architecture model consisting of the Presentation Layer, Application Layer, and Data Layer. This architecture separates the user interface, business logic, and data management responsibilities into independent layers, improving maintainability, scalability, modularity, and system organization.

The architecture is specifically designed to support cross-platform mobile operation, offline-first functionality, secure synchronization, modular feature development, maintainability and scalability, separation of concerns, and efficient local and remote data management.

4.2.1 Architectural Layers

Presentation Layer

The presentation layer consists of the Flutter mobile application running on Android and iOS devices. Its responsibilities include displaying user interfaces, accepting user input, providing charts and visualizations, performing local validation, managing local interactions, and triggering synchronization requests.

Application Layer

The application layer contains the backend REST API and business logic implemented using Golang. Its responsibilities include authentication and authorization, business rule processing, synchronization handling, data validation, notification management, conflict resolution, and API request handling.

Data Layer

The data layer consists of SQLite and PostgreSQL databases. Its responsibilities include local offline data storage, centralized server storage, data persistence, synchronization tracking, backup and recovery, and query processing.

4.2.2 Architectural Workflow

The overall operational workflow of the system is as follows:

1. The user interacts with the Flutter mobile application.
2. User input is validated locally.
3. Data is immediately stored in the SQLite local database.
4. If internet connectivity exists:
   - Data is sent to the backend REST API.
   - The backend validates the request.
   - Data is stored in PostgreSQL.
   - Synchronization status is updated.
5. If internet connectivity is unavailable:
   - Data remains locally stored.
   - Records are marked as pending synchronization.
6. Once connectivity becomes available:
   - Pending records are synchronized automatically.

This architecture ensures uninterrupted system functionality in both online and offline environments.

4.2.3 Advantages of the Proposed Architecture

The selected architecture provides several advantages:

Table 4-1: Advantages of the Proposed Architecture

Advantage	**Description**
Offline Capability	Users can continue using the application without internet access.
Scalability	The backend can support future feature expansion and additional users.
Maintainability	Clear separation of layers simplifies debugging and updates.
Cross-Platform Support	Flutter enables Android and iOS support using one codebase.
Reliability	Local storage prevents data loss during connectivity interruptions.
Security	Sensitive operations are managed securely through the backend API.

4.3 System Architecture Diagram

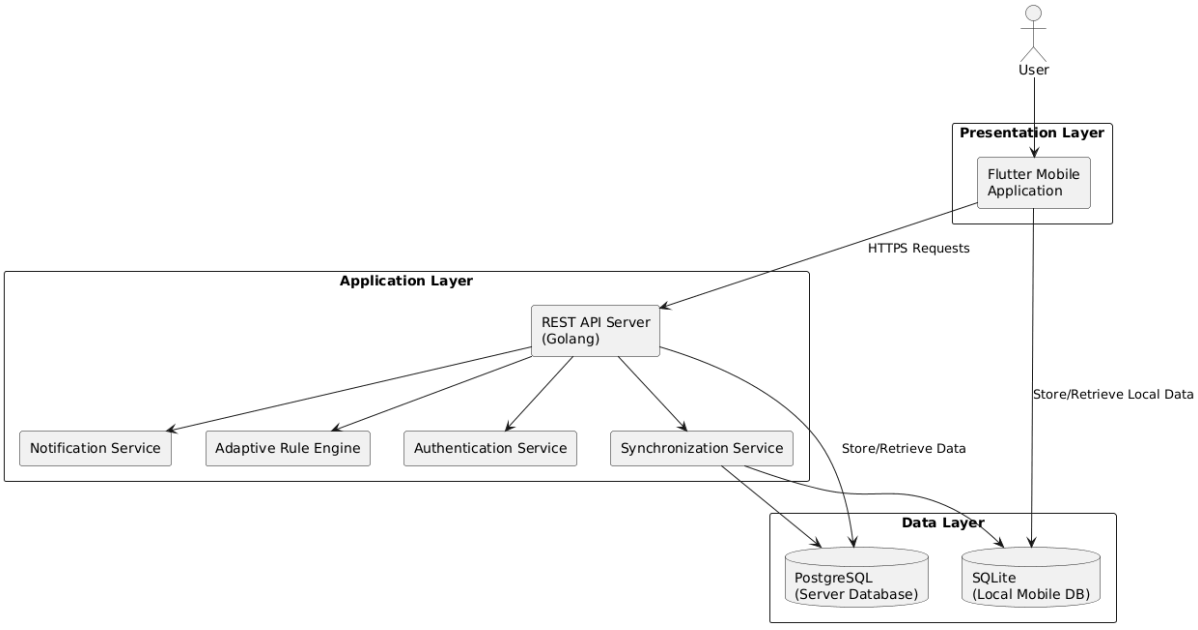


Figure 4-1: System Architecture Diagram

Architectural Components

Mobile Frontend Layer

The frontend layer is implemented using Flutter and Dart. It provides all user-facing interfaces and handles user interactions. Its responsibilities include user authentication screens, expense management interfaces, debt tracking interfaces, dashboard visualization, local data validation, offline data storage, local notifications, and synchronization requests. The advantages of Flutter are a single codebase for Android and iOS, fast UI rendering, native-like performance, a rich widget ecosystem, reduced development time, and strong community support.

Backend Application Layer

The backend is implemented using Golang and a RESTful API architecture. Its responsibilities include authentication and authorization, password hashing and verification, centralized data storage, data synchronization, validation of API requests, business rule processing, notification processing, and conflict resolution during synchronization. The advantages of Golang are high performance, efficient concurrency handling, a lightweight runtime, strong support for REST APIs, and simplicity and maintainability.

Data Layer

The system uses two databases: SQLite for the local mobile database and PostgreSQL for the remote server database. SQLite is responsible for storing expenses locally, storing debts locally, queuing unsynchronized changes, storing reminders and notifications, and maintaining application functionality while offline. Its advantages are that it is lightweight, embedded within the mobile application, fast for local operations, and has no internet dependency. PostgreSQL acts as the centralized server-side database. Its responsibilities include permanent cloud storage, multi-device synchronization, backup and recovery, user account management, and centralized reporting. The advantages of PostgreSQL are reliability, ACID compliance, strong relational capabilities, scalability, and advanced indexing and querying.

4.4 System Component Design

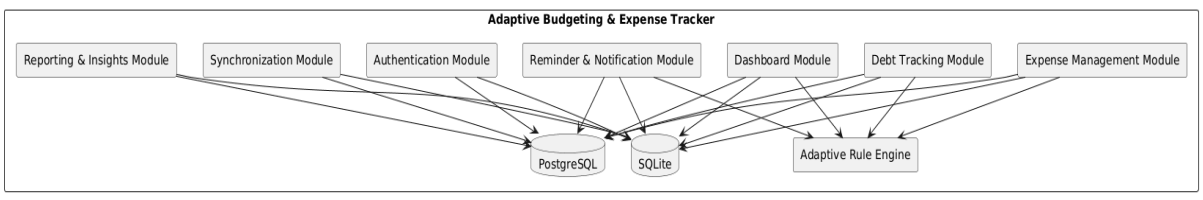


Figure 4-2: System Component Diagram

The application architecture is divided into several specialized components that work together to manage the user's financial lifecycle. By modularizing the system, each component maintains a specific scope of logic, ranging from secure data access to financial analysis and cross-device synchronization.

4.4.1 Authentication Component

The Authentication Component serves as the primary security layer of the system. Its main responsibility is to manage the user lifecycle, which includes account registration, credential validation, and session maintenance. During registration, the system ensures data integrity through strict input validation and secures user credentials by hashing passwords using the **bcrypt** algorithm. The component also handles secure login flows, generates tokens for password resets, and manages active user sessions to ensure that financial data remains private. To maintain a high security standard, this module enforces HTTPS communication, utilizes session timeouts, and performs comprehensive sanitization on all user inputs to prevent unauthorized access or injection attacks.

4.4.2 Expense Management Component

At the core of the application's functionality is the Expense Management Component, which facilitates the tracking of daily financial transactions. This component is responsible for the full range of CRUD (Create, Read, Update, and Delete) operations, allowing users to accurately record their spending. When a user inputs an amount, category, date, and description, the component validates the data types and ensures structural integrity before persisting the record to the local database. Beyond simple storage, it provides the logic for multidimensional filtering and search, enabling users to view their spending habits through various lenses. It also feeds data into the reporting engine to generate real-time spending summaries.

4.4.3 Debt Tracking Component

The Debt Tracking Component is designed to manage peer-to-peer financial obligations by distinguishing between money borrowed (liabilities) and money lent (assets). This module allows users to maintain a clear ledger of counterparties, principal amounts, and due dates. It supports advanced features such as partial repayment tracking and automated settlement logic to ensure that balances are always up to date. To help users maintain their financial reputation, the component integrates with the system's notification service to schedule and

trigger reminders for upcoming or overdue payments. This ensures that users have a comprehensive view of their total debt standing at all times.

4.4.4 Dashboard Component

The Dashboard Component acts as the central visualization hub, aggregating data from the expense and debt modules to provide a high-level financial overview. It is responsible for calculating and displaying critical metrics, such as total monthly spending, the percentage of the budget currently consumed, and the distribution of costs across different categories. By utilizing a simple moving average of historical data, the dashboard also presents spending predictions and highlights recent transactions. This component ensures that the most relevant information—including active alerts and upcoming reminders—is immediately accessible to the user upon opening the application.

4.4.5 Adaptive Features Component

To provide proactive financial guidance, the Adaptive Features Component implements a rule-based engine that monitors user behaviour. Unlike complex machine learning models, this component uses a deterministic **IF-THEN logic** that is highly efficient for mobile environments and functions entirely offline. For example, if the system detects that food expenditures have exceeded 40% of the total monthly spending, it automatically generates a recommendation for the user to reduce spending in that category. This "intelligent" layer provides budget alerts, detects unusual spending patterns, ensuring the user receives predictable and helpful financial coaching without high computational requirements.

4.4.6 Synchronization Component

The Synchronization Component ensures that data remains consistent between the local mobile storage and the remote cloud database. This process begins by detecting an active internet connection and auditing the state of local versus remote records using timestamps. The component is responsible for uploading pending local changes and downloading any updates made on other devices. In the event of a conflict—such as a record being edited in two places—the system employs a **"Server Wins"** strategy, where the remote server's data is treated as the authoritative version. This module maintains detailed logs of every sync attempt and includes built-in retry logic to handle intermittent connectivity issues gracefully.

4.5 Database Design Overview

The database is designed to support both offline (SQLite) and online (PostgreSQL) operation using a consistent relational schema. The schema follows Third Normal Form (3NF) to minimize redundancy, maintain data consistency, and improve storage efficiency.

The design supports offline-first functionality through synchronization tracking fields such as is_synced and sync_status. SQLite is used as the local embedded database for offline mobile operations, while PostgreSQL is used as the centralized server-side database for synchronization, backup, and persistent cloud storage.

The database stores and manages user information, expense records, peer-to-peer debt records, categories, reminders, and synchronization logs.

4.5.1 Entity Relationship Design

The following entities are defined within the database schema. Primary keys are denoted by <PK> and foreign keys by <FK>. Attribute types are aligned with the class diagram and implementation requirements.

Table 4-2: User Entity Attributes

Attribute	**Type**	**Description**
user_id	int <PK>	Unique identifier
email	String	Unique login email
password_hash	String	Hashed password
full_name	String	Display name
monthly_budget	double	Monthly spending limit
created_at	DATETIME	Account creation timestamp
last_sync_timestamp	DATETIME	Last successful synchronization

Table 4-3: Category Entity Attributes

Attribute	**Type**	**Description**
category_id	int <PK>	Unique identifier
user_id	int <FK>	Owner user (null for predefined categories)
name	String	Category name
icon	String	Emoji or icon identifier
is_predefined	boolean	Indicates system-defined category

Table 4-4: Expense Entity Attributes

Attribute	**Type**	**Description**
expense_id	int <PK>	Unique identifier
user_id	int <FK>	References User
category_id	int <FK>	References Category
amount	double	Expense amount
expense_date	DATE	Transaction date
description	String	Optional note
is_synced	boolean	Synchronization confirmation flag
sync_status	String	Synchronization state (pending or synced)

Table 4-5: Debt Entity Attributes

Attribute	**Type**	**Description**
debt_id	int <PK>	Unique identifier
user_id	int <FK>	References User
counterparty_name	String	Name of the other person
amount	double	Original debt amount
is_lent	boolean	True if others owe the user
remaining_amount	double	Outstanding balance
due_date	DATE	Optional repayment deadline
status	String	pending, partial, or settled
notes	String	Additional remarks
is_synced	boolean	Synchronization flag

Table 4-6: Reminder Entity Attributes

Attribute	**Type**	**Description**
reminder_id	int <PK>	Unique identifier
user_id	int <FK>	References User
expense_id	int <FK>	Optional reference to an Expense
debt_id	int <FK>	Optional reference to a Debt

title	String	Reminder message
reminder_datetime	DATETIME	Reminder trigger timestamp
recurrence_pattern	String	none, daily, weekly, or monthly
is_active	boolean	Indicates whether reminder is enabled
last_triggered	DATETIME	Last notification timestamp

Table 4-7: SyncLog Entity Attributes

Attribute	**Type**	**Description**
sync_id	int <PK>	Unique identifier
user_id	int <FK>	References User
last_sync_at	DATETIME	Synchronization timestamp
records_uploaded	int	Number of uploaded records
records_downloaded	int	Number of downloaded records
status	String	success, partial, or failed

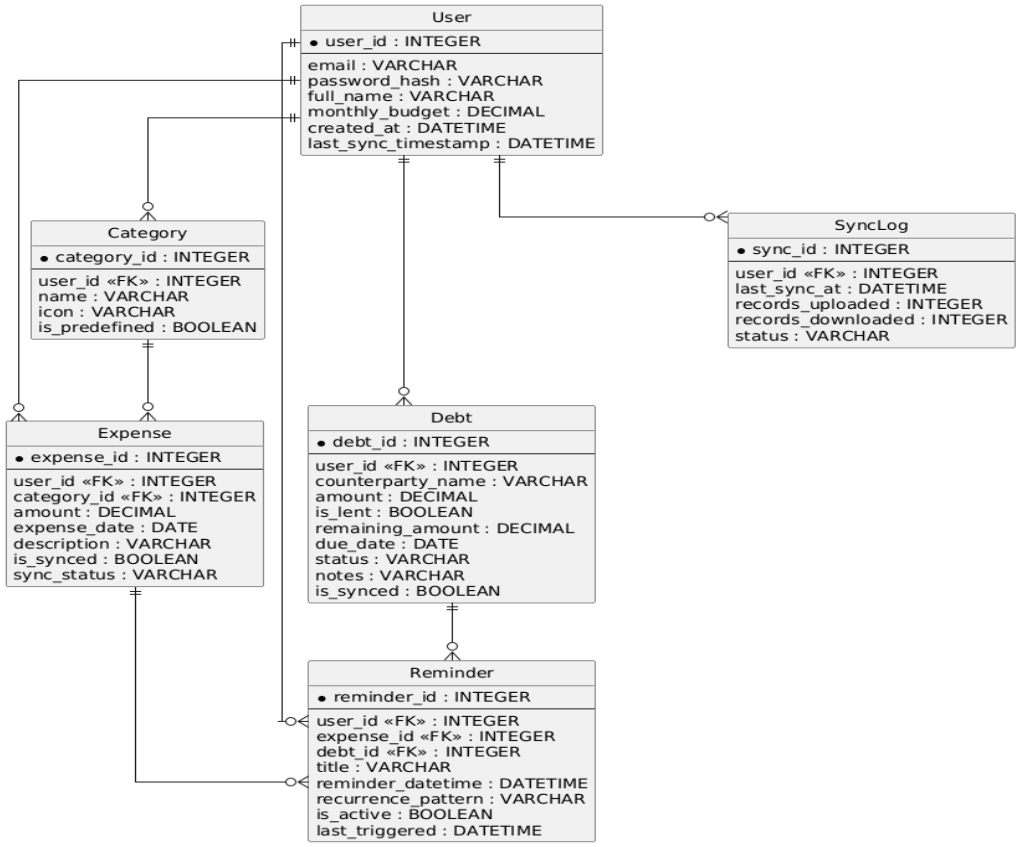


Figure 4-3: Entity-Relationship Diagram (ERD)

4.5.2 Relationships and Integrity

A User can have multiple Expenses, Debts, Reminders, SyncLog entries, and user-created Categories. Each Expense belongs to one User and one Category, while a Category may classify multiple Expenses.

A Reminder may optionally reference either an Expense or a Debt depending on its purpose. Expense-related reminders are used for recurring spending notifications, while debt-related reminders are used for repayment tracking.

Referential integrity constraints are enforced throughout the schema. Deleting a User cascades to all related records. Deleting a Category is restricted if expenses are still associated with it, requiring reassignment before removal. Deleting an Expense or Debt sets the related foreign key in Reminder to null in order to preserve reminder history.

The schema supports efficient querying, minimizes redundancy, and ensures consistency between local and remote databases.

4.6 Offline Synchronisation

The system follows an offline-first synchronization strategy. All records are initially stored in the local SQLite database regardless of internet availability.

Expense and Debt records contain synchronization tracking fields including is_synced and sync_status. When the device is offline, newly created records are stored locally with is_synced = false and sync_status = 'pending'.

Once internet connectivity becomes available, the system uploads all pending records to the PostgreSQL server database and downloads remote changes created after the user's last_sync_timestamp.

Synchronization conflicts are resolved using a server-wins strategy to maintain consistency across devices and prevent divergent records. Each synchronization attempt is recorded within the SyncLog table for auditing and monitoring purposes.

This synchronization mechanism ensures continuous system availability, minimizes data loss risk, and supports reliable operation in environments with unstable internet connectivity.

4.7 User Interface Design

The User Interface (UI) design focuses on providing a seamless and intuitive experience for users to manage their personal finances. The interface is designed to be "mobile-first," ensuring that essential tasks like recording expenses and checking debt balances can be

performed with minimal effort. The design balances aesthetic appeal with functional clarity, following modern Material Design standards.

Design Principles

To ensure high usability and user satisfaction, the following principles were applied throughout the design process:

- **Simplicity:** The interface is streamlined to ensure that primary tasks—such as adding a new transaction—require minimal interaction and cognitive load.
- **Consistency:** Visual elements including buttons, color palettes, icons, and layout structures remain uniform across all screens to reduce the learning curve.
- **Accessibility:** Readable font sizes, high-contrast text, and clear labels are used to improve usability for a wide range of users.
- **Feedback:** The system provides immediate visual feedback through confirmation dialogs, alerts, and loading indicators to keep the user informed of the application's state.
- **Responsiveness:** The layout utilizes flexible widgets to ensure the interface adapts gracefully to various mobile screen sizes and aspect ratios.

4.7.1 Main Screen Specifications

Login Screen

- **Features:** Email input, Password input, Login button, "Forgot Password" link, and Navigation to registration.
- **Purpose:** To securely authenticate existing users and provide access to their synchronized financial data.

Registration Screen

- **Features:** Full name field, Email field, Password and Confirm Password fields, and Register button.
- **Purpose:** To allow new users to create an account and initialize their personal budget settings.

Dashboard Screen

- **Features:** Total spending summary, Budget progress bar, Recent expenses list, Highest spending category indicator, Prediction section, and Alerts/Notifications.
- **Purpose:** To act as the central hub, providing a quick financial overview and immediate insight into budget health.

Add Expense Screen

- **Features:** Amount field, Category dropdown, Date picker, Description field, and Save button.
- **Purpose:** To facilitate the rapid and accurate recording of daily expenditures.

Debt Management Screen

- **Features:** "Borrowed" and "Lent" toggle tabs, "Add Debt" button, searchable debt lists, and settlement action buttons.
- **Purpose:** To manage peer-to-peer financial obligations and track the settlement status of various debts.

Insight Screen

- **Features:** Interactive pie charts for category distribution, trend graphs for monthly spending, and comparative analysis tools.
- **Purpose:** To visualize financial behavior over time, helping users identify spending patterns and areas for potential savings.

Reminder Screen

- **Features:** List of upcoming reminders, "Add Reminder" button, and notification frequency settings.
- **Purpose:** To ensure users stay informed about upcoming bill payments, debt due dates, and budget milestones.

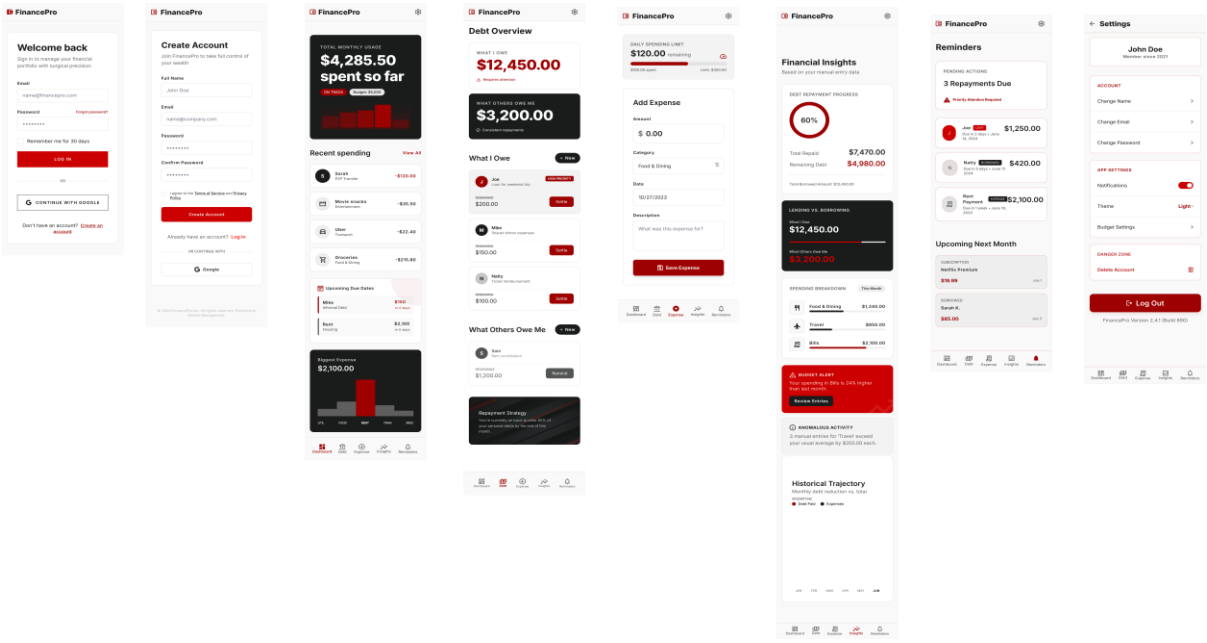


Figure 4-4: User Interface Screens

4.8 API Design

Overview

The backend exposes RESTful API endpoints for communication between the mobile application and server.

All APIs use JSON format.

Base URL Exmaple: /api/v1/

Table 4-8: Authentication API Endpoints

Endpoint	**Method**	**Purpose**
/register	POST	Create account
/login	POST	Authenticate user
/forgot-password	POST	Request reset
/reset-password	POST	Update password
/profile	PUT	Update profile

Table 4-9: Expense Management API Endpoints

Endpoint	**Method**	**Purpose**
/expenses	GET	Retrieve expenses
/expenses	POST	Add expense
/expenses/{id}	PUT	Edit expense
/expenses/{id}	DELETE	Delete expense

Table 4-10: Debt Management API Endpoints

Endpoint	**Method**	**Purpose**
/debts	GET	Retrieve debts
/debts	POST	Add debt
/debts/{id}	PUT	Edit debt
/debts/{id}	DELETE	Delete debt
/debts/{id}/settle	PATCH	Settle debt

Table 4-11: Synchronization API Endpoints

Endpoint	Method	Purpose
/sync/upload	POST	Upload local changes
/sync/download	GET	Download remote changes
/sync/status	GET	Retrieve sync status

4.9 Security Design

Overview

Security is a fundamental requirement for the application due to the sensitive nature of the financial information being processed. To protect user data from unauthorized access and ensure the integrity of the system, a multi-layered security strategy has been implemented, covering authentication, data communication, and local storage.

Authentication Security

The system employs several measures to secure user accounts. All user passwords are processed using the **bcrypt** hashing algorithm before storage in the database. This approach ensures that plain text passwords are never stored, providing high resistance against rainbow table and brute-force attacks. Furthermore, the system incorporates robust session management where authenticated sessions are configured to expire after periods of inactivity. For users who lose access to their accounts, a secure password reset workflow is utilized, relying on temporary, high-entropy verification tokens to ensure that only the rightful owner can regain access.

Communication Security

To ensure the privacy and integrity of data in transit, all client-server interactions are conducted exclusively over **HTTPS** utilizing Transport Layer Security (TLS). This encryption prevents the interception of sensitive credentials and transaction data by third parties, effectively mitigating man-in-the-middle (MITM) attacks and ensuring that the data received by the server remains identical to the data sent by the client.

Input Validation

The backend architecture enforces strict input validation protocols to maintain system stability and security. Every incoming request is scrutinized for correct email formatting, appropriate numeric ranges, and the presence of all required fields. By filtering out invalid characters and malformed requests, the system prevents common vulnerabilities such as SQL

injection and unauthorized data manipulation, ensuring that only clean, predictable data enters the database.

Local Data Security

On the mobile device, the application leverages native security features to protect stored information. This includes device-level file protection and application sandboxing, which prevents other installed apps from accessing the financial database. While the current version relies on secure session handling and native protections, future iterations are planned to include encrypted SQLite storage for an additional layer of local data security.

Design Considerations

Scalability

The backend architecture is designed with future expansion in mind. The modular nature of the system allows for the seamless addition of features such as a web-based dashboard, multi-currency support, and AI-based financial predictions. Maintainability is achieved through a strict separation of layers and the use of reusable components. Adherence to consistent coding standards and the maintenance of comprehensive API documentation ensures that the system can be easily updated or scaled by different development teams.

Maintainability

System reliability is supported by the offline-first approach and the use of detailed synchronization logs and automatic backups. Performance is optimized by conducting heavy operations on the local SQLite database and utilizing lightweight REST APIs for data transfer. Furthermore, the use of indexed database queries and efficient rendering in Flutter ensures that the user interface remains responsive and fluid, even as the volume of financial data grows.

Reliability

The reliability of the system is anchored in its "offline-first" architecture, which ensures that the application remains fully functional regardless of the user's network status. By utilizing local SQLite storage as the primary data interface, the system eliminates the risk of data loss or app crashes during intermittent connectivity. To further enhance system integrity, comprehensive synchronization logs are maintained to track data state and history, while automated backup routines protect against local device failure. The software also incorporates robust error-handling mechanisms that gracefully manage unexpected input or server downtime. Additionally, background synchronization allows the app to reconcile data silently

in the secondary execution thread, ensuring that the local and remote databases stay consistent without interrupting the user experience.

Performance

Performance optimization is achieved through a multi-layered approach that targets both the client and server environments. On the mobile device, the application prioritizes local SQLite operations to ensure near-instantaneous response times for data entry and retrieval. To further speed up data access, all critical database tables utilize indexed queries, significantly reducing search and filtering latency as the user's transaction history grows. On the networking side, the system uses lightweight REST APIs designed to minimize payload sizes and reduce mobile data consumption. Finally, the user interface leverages Flutter’s high-performance rendering engine to provide fluid transitions and 60 FPS animations, ensuring that the visual experience remains smooth even during complex data visualizations or heavy background processing.

References

- Collins, D., Morduch, J., Rutherford, S., & Ruthven, O. (2009). *Portfolios of the Poor: How the World's Poor Live on $2 a Day.* Princeton University Press.
- GSMA. (2023). *The Mobile Economy Sub-Saharan Africa 2023.* GSMA Intelligence.
- Lusardi, A., & Mitchell, O. S. (2014). *The Economic Importance of Financial Literacy.* Journal of Economic Literature.
- Worku, T. (2022). *Financial Behaviour of Ethiopian University Students.* Addis Ababa University.